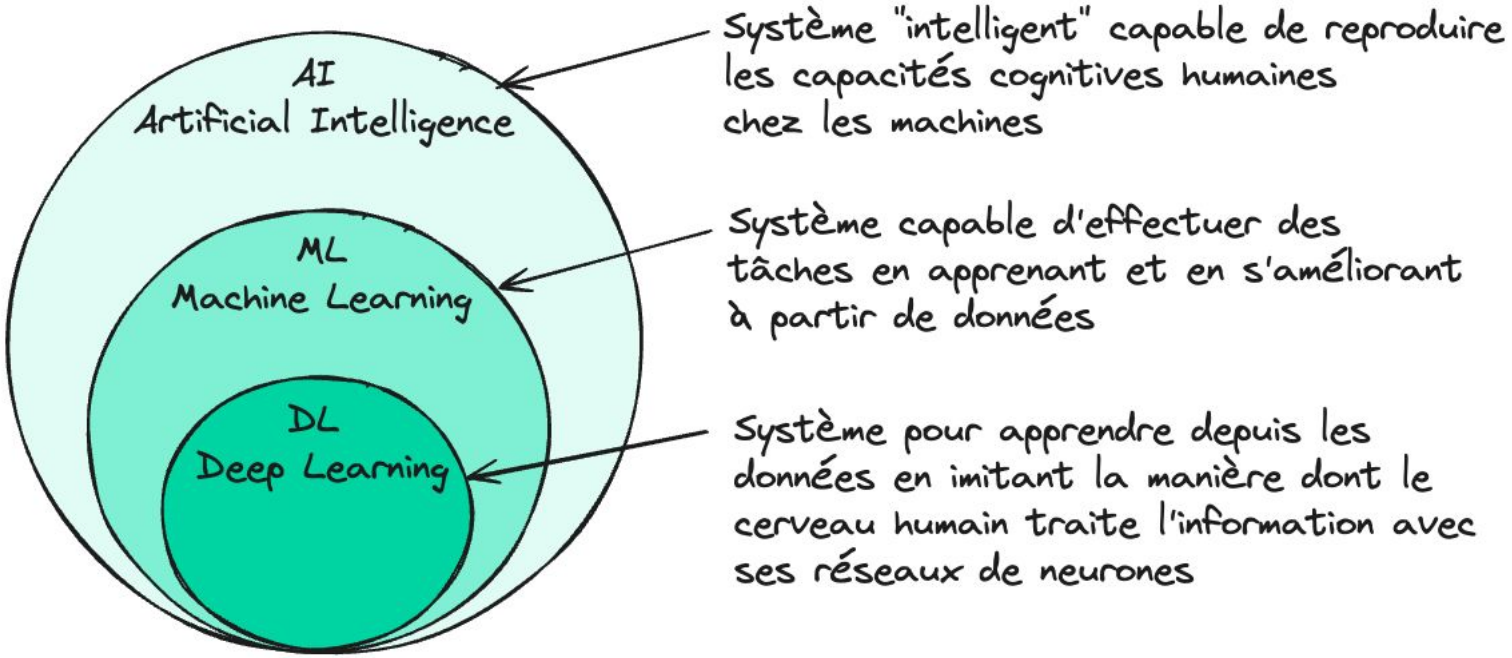
Fondamentaux de l'Intelligence Artificielle

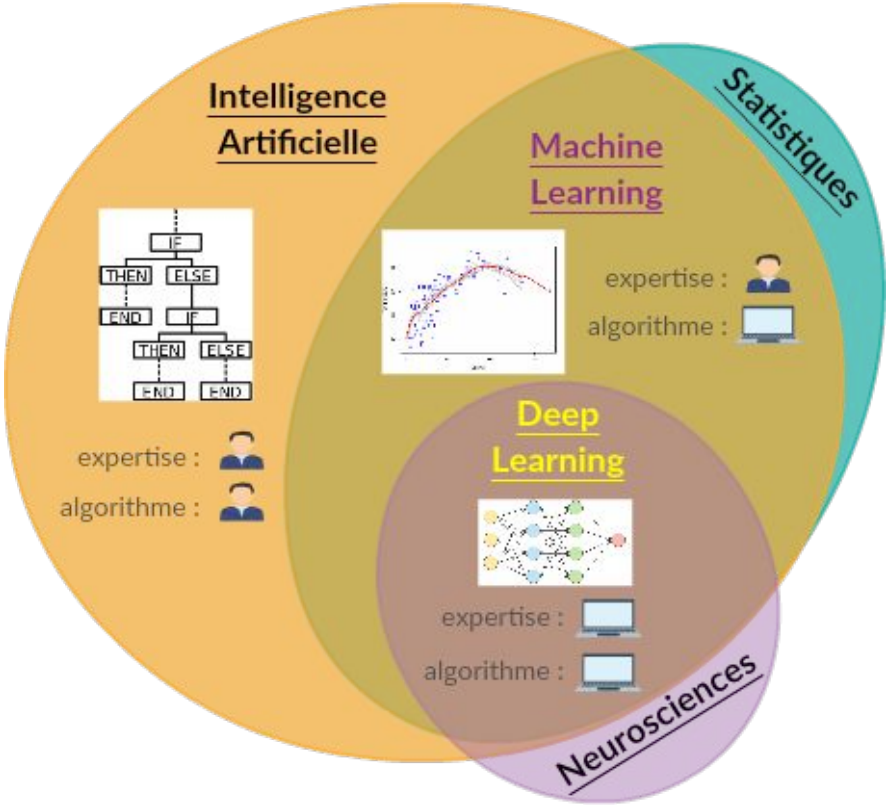
et du Traitement Automatique du Langage Naturel

Qu’est-ce que l’intelligence artificielle ?

Qu’est-ce que l’intelligence artificielle ?

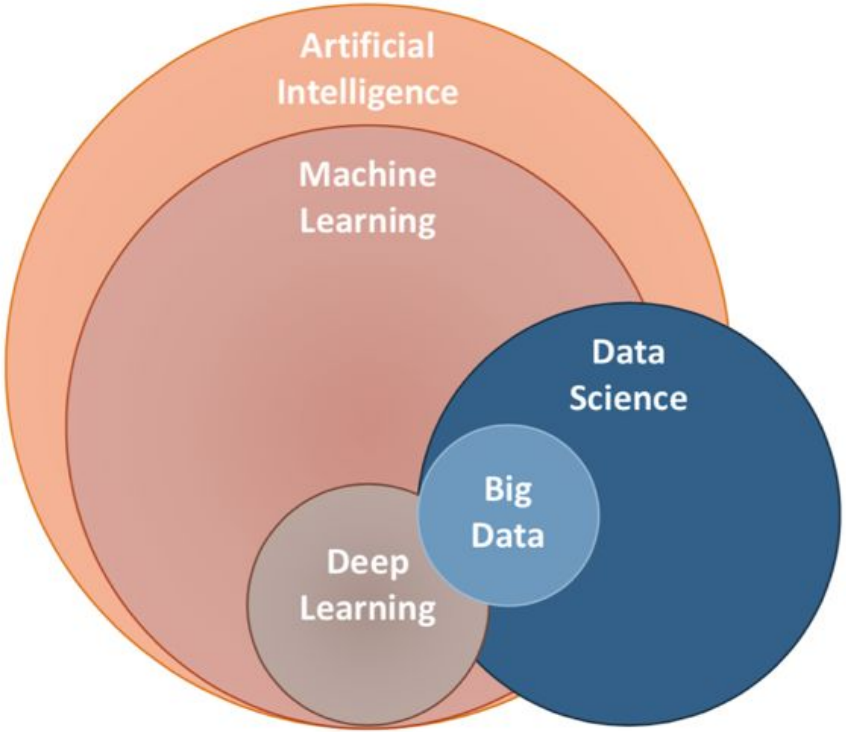


Qu’est-ce que l’intelligence artificielle ?

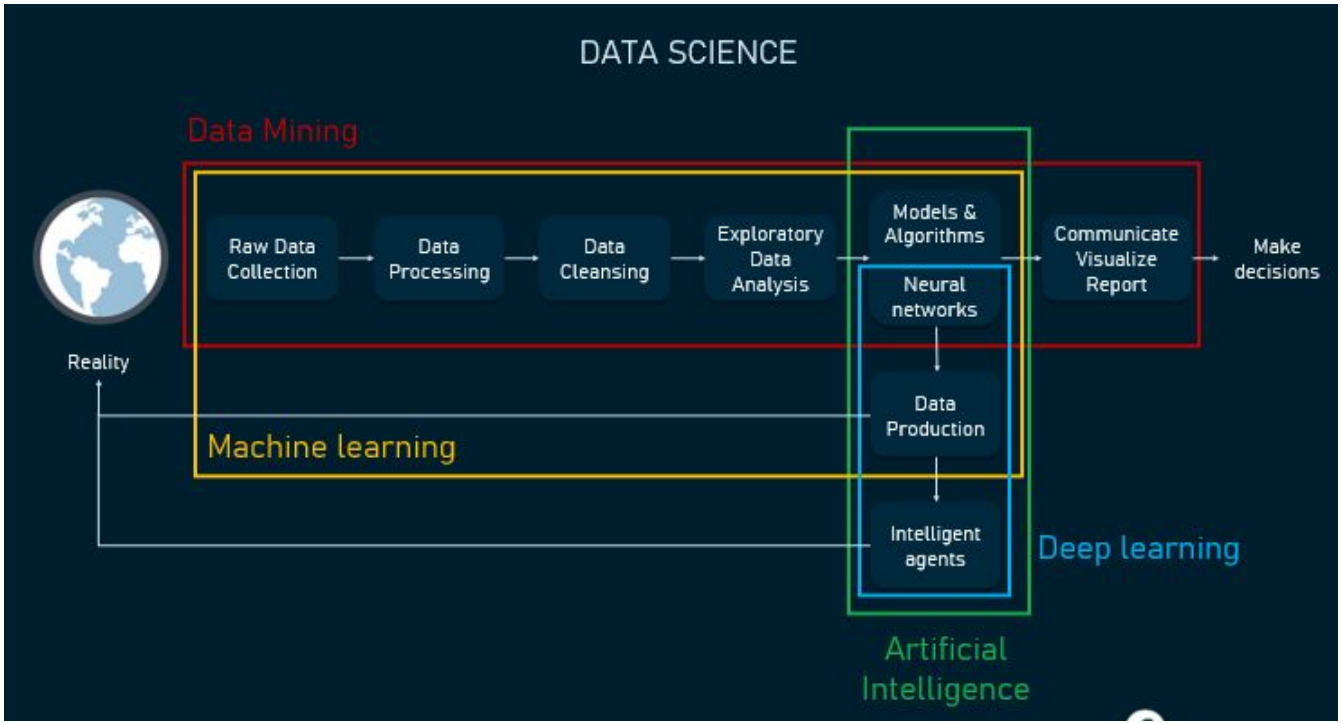


Qu’est ce que la data science

Qu’est ce que la data science



Qu’est ce que la data science



Démonstration

1. Félicitations ! Vous venez d’être recruté

pour conduire les efforts en faveur de la data science chez DataSciencester, le réseau social des experts du domaine.

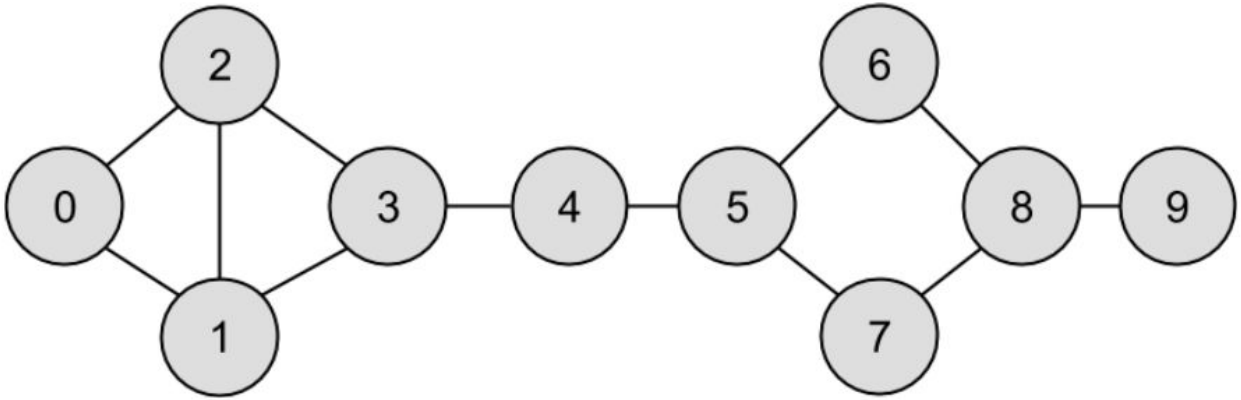
identifiez les « connecteurs » parmi les experts en DS

la responsable de la Fraternisation débarque dans votre bureau. Elle veut encourager davantage de connexions au sein de vos relations.

identifiez les « connecteurs » parmi les experts en DS

Il vous remet également les « relations » entre les inscrits, représentées par une liste de paires d'id comme suit :

```
friendship_pairs = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 3),
    (3, 4), (4, 5), (5, 6), (5, 7), (6, 8), (7, 8), (8, 9)]
```



identifiez les « connecteurs » parmi les experts en DS

Pour créer le dict, il faut, bien sûr, examiner chaque paire, mais une seule fois. Les consultations suivantes seront bien moins coûteuses :

```
# initialiser le dict avec une liste vide pour chaque id
d'utilisateur
friendships = {user["id"]: [] for user in users}
# et boucler sur les paires de relation pour le remplir
for i, j in friendship_pairs: friendships[i].append(j)
    # ajoute j comme ami de i friendships[j].append(i)
    # ajoute i comme ami de j
```

identifiez les « connecteurs » parmi les experts en DS

Une fois que chaque dict contient sa liste d'amis, il est facile pour nous de répondre à des questions comme « Quel est le nombre moyen de connexions ? »

```
def number_of_friends(user):
    """combien d'amis l'utilisateur _user_ a-t-il ?"""
    user_id = user["id"]
    friend_ids = friendships[user_id]
    return len(friend_ids)

total_connections = sum(number_of_friends(user)
                        for user in users) # 24
```

identifiez les « connecteurs » parmi les experts en DS

Il suffit ensuite de diviser par le nombre total d'utilisateurs :

```
num_users = len(users) # longueur de la liste friend_ids
avg_connections = total_connections / num_users # 24 / 10 == 2.4
```

identifiez les « connecteurs » parmi les experts en DS

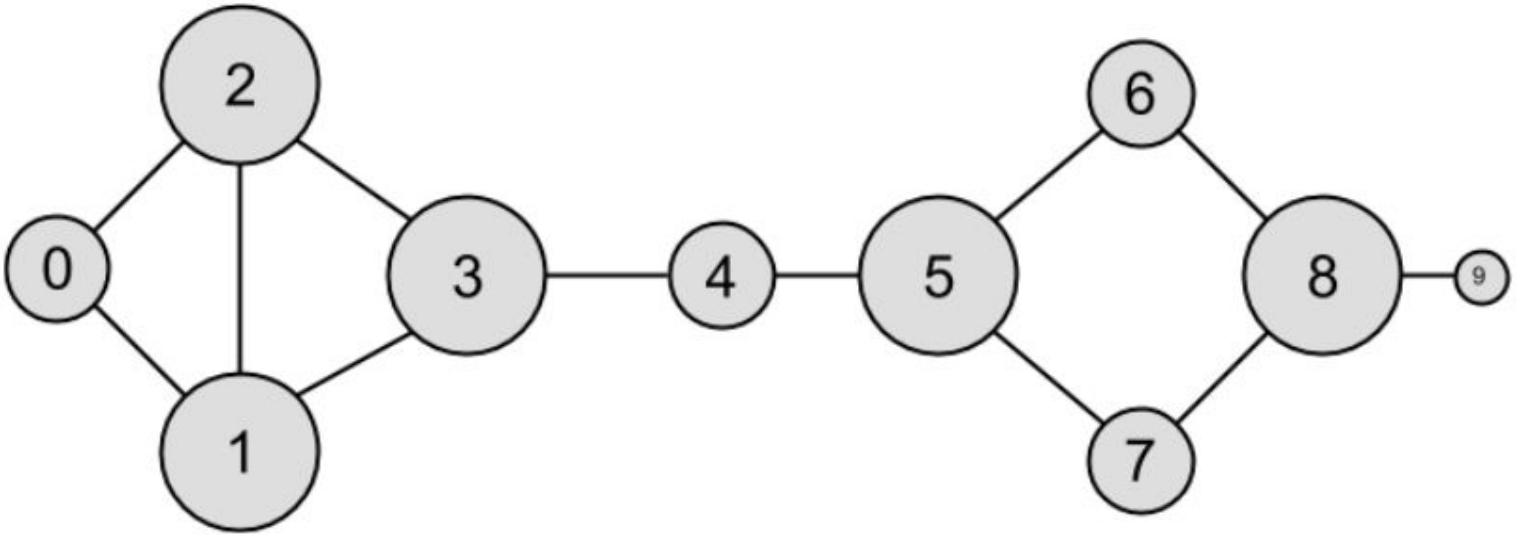
Comme il n’y a pas beaucoup d’utilisateurs, nous pouvons les trier depuis « le plus d’amis » jusqu’à « le moins d’amis » :

```
# créer une liste (user_id, number_of_friends)
num_friends_by_id =[(user["id"], number_of_friends(user)) for user in users]
num_friends_by_id.sort( # trier la liste
key=lambda id_and_friends: id_and_friends[1], # suivant num_friends
                        reverse=True) # du plus grand au plus petit

# chaque paire vaut (user_id, num_friends)
# [(1, 3), (2, 3), (3, 3), (5, 3), (8, 3),
# (0, 2), (4, 2), (6, 2), (7, 2), (9, 1)]
```

identifiez les « connecteurs » parmi les experts en DS

On peut considérer que c'est une façon d'identifier les personnes centrales du réseau. En fait, nous venons de calculer le degré de centralité du réseau :



un système de suggestion d’amis

```
def foaf_ids_bad(user):
    """foaf" est l’abréviation de "friend of a friend",
    autrement dit "ami d’un ami""""
    return [foaf_id
        for friend_id in friendships[user["id"]]
        for foaf_id in friendships[friend_id]]
```

Quand nous appelons cette fonction sur users[0] (Hero), elle récupère les informations suivantes :

[0, 2, 3, 0, 1, 3]

un système de suggestion d’amis

```
print(friendships[0]) # [1, 2]

print(friendships[1]) # [0, 2, 3]

print(friendships[2]) # [0, 1, 3]
```

un système de suggestion d’amis

```
from collections import Counter # n'est pas chargé par défaut

def friends_of_friends(user): user_id = user["id"]
    return Counter(
        foaf_id
        for friend_id in friendships[user_id] # pour chacun de mes amis
        for foaf_id in friendships[friend_id] # trouve leurs amis
        if foaf_id != user_id # qui ne sont pas moi
        and foaf_id not in friendships[user_id] # et ne sont pas mes
amis )

print(friends_of_friends(users[3])) # compteur({0: 2, 5: 1})
```

un système de suggestion d’amis

En tant que data scientist, vous aimeriez aussi rencontrer des utilisateurs qui partagent vos centres d’intérêt (un bon exemple de « domaine de prédilection » de la data science).

vous avez réussi à mettre la main sur ces données, sous la forme d’une liste de paires (user_id, interest) :

un système de suggestion d’amis

```
interests = [
    (0, "Hadoop"), (0, "Big Data"), (0, "HBase"), (0, "Java"),
    (0, "Spark"), (0, "Storm"), (0, "Cassandra"), (1, "NoSQL"),
    (1, "MongoDB"), (1, "Cassandra"), (1, "HBase"), (1, "Postgres"),
    (2, "Python"), (2, "scikit-learn"), (2, "scipy"), (2, "numpy"),
    (2, "statsmodels"), (2, "pandas"), (3, "R"), (3, "Python"),
    (3, "statistics"), (3, "regression"), (3, "probability"),
    (4, "machine learning"), (4, "regression"),
    (4, "decision trees"), (4, "libsvm"), (5, "Python"), (5, "R"), (5, "Java"), (5, "C++"), (5, "Haskell"),
    (5, "programming languages"), (6, "statistics"),
    (6, "probability"), (6, "mathematics"), (6, "theory"),
    (7, "machine learning"), (7, "scikit-learn"), (7, "Mahout"),
    (7, "neural networks"), (8, "neural networks"),
    (8, "deep learning"), (8, "Big Data"),
    (8, "artificial intelligence"), (9, "Hadoop"), (9, "Java"),
    (9, "MapReduce"), (9, "Big Data")
]
```

un système de suggestion d’amis

Il est facile de construire une fonction qui trouve les utilisateurs présentant un centre d’intérêt commun :

```
def data_scientists_who_like(target_interest):
    """trouve tous les utilisateurs partageant le centre d'intérêt
cible"""
    return [user_id
        for user_id, user_interest in interests
        if user_interest == target_interest]
```

Cela fonctionne, mais il faut passer en revue la liste complète des centres d’intérêt à chaque recherche.

un système de suggestion d’amis

il sera sans doute préférable de construire un index des centres d’intérêt par utilisateur :

```
from collections import defaultdict
# La clé est le centre d’intérêt, la valeur est la liste
# d’identifiants user_id ayant ce centre d'intérêt.
user_ids_by_interest = defaultdict(list)
for user_id, interest in interests:
    user_ids_by_interest[interest].append(user_id)
```

Cela fonctionne, mais il faut passer en revue la liste complète des centres d’intérêt à chaque recherche.

un système de suggestion d’amis

Et un autre à partir des utilisateurs vers les centres d’intérêt :

```
# La clé est l’identifiant user_id, la valeur est la liste
# de centres d’intérêt pour ce user_id.
interests_by_user_id = defaultdict(list)
for user_id, interest in interests:
    interests_by_user_id[user_id].append(interest)
```

un système de suggestion d’amis

trouver qui a le plus de centres d’intérêt en commun avec un utilisateur donné :

```
def most_common_interests_with(user):
    return Counter(
        interested_user_id
        for interest in interests_by_user_id[user["id"]]
        for interested_user_id in user_ids_by_interest[interest]
        if interested_user_id != user["id"]
    )
```

Illustrer

illustrer de manière ludique la question du salaire d’un expert en data science

```
salaries_and_tenures = [(83000, 8.7), (48000,0.7), (69000, 6.5),
                        (60000, 2.5), (48000,1.9), (88000, 8.1),
                        (76000, 6),  (76000, 7.5),  (83000, 10),
                        (63000, 4.2)]
```

Illustrer

```
# La clé est l’année, la valeur est la liste de salaires # pour chaque période d'affectation.
salary_by_tenure = defaultdict(list)
for salary, tenure in salaries_and_tenures:
    salary_by_tenure[tenure].append(salary)
    # La clé est l’année, la valeur est le salaire moyen
    # pour cette période.
     average_salary_by_tenure = {
        tenure: sum(salaries) / len(salaries)
        for tenure, salaries in salary_by_tenure.items()
    }
```

Illustrer

```
{0.7: 48000.0, 1.9: 48000.0,
2.5: 60000.0, 4.2: 63000.0,
6: 76000.0, 6.5: 69000.0,
7.5: 76000.0, 8.1: 88000.0,
8.7: 83000.0, 10: 83000.0}
```

Illustrer

Il serait sans doute plus intéressant de répartir l'ancienneté en catégories :

```
def tenure_bucket(tenure):
      if tenure < 2:
    return "less than two" elif tenure < 5:
    return "between two and five" else:
    return "more than five"
```

Pour ensuite regrouper les salaires dans les différentes catégories :

```
# La clé est la catégorie de poste, la valeur est la liste
# de salaires pour cette catégorie.
salary_by_tenure_bucket = defaultdict(list)
for salary, tenure in salaries_and_tenures:
    bucket = tenure_bucket(tenure)
    salary_by_tenure_bucket[bucket].append(salary)
```

Illustrer

Et enfin, calculer le salaire moyen de chaque catégorie :

```
# La clé est la catégorie de poste, la valeur est le salaire moye pour cette catégorie.
average_salary_by_bucket = {
    tenure_bucket: sum(salaries) / len(salaries)
    for tenure_bucket, salaries
        in salary_by_tenure_bucket.items()
}
```

Le résultat est plus intéressant :

```
{'between two and five': 61500.0,
'less than two': 48000.0,
'more than five': 79166.66666666667}
```

Illustrer

Cependant, nous avons choisi les catégories de manière totalement arbitraire.

Nous préférerions dégager une règle sur l'effet que peut avoir une année d'ancienneté supplémentaire sur le salaire (en moyenne).

En plus de nous procurer une annonce plus percutante, cela nous permettrait de faire des prévisions sur des salaires que nous ne connaissons pas.

Nous développerons cette idée au ***chapitre 14.***

Prédire

mieux appréhender quels utilisateurs paient pour avoir un compte et lesquels ne paient pas.
Vous remarquez qu'il semble exister une correspondance entre l'expérience et les comptes payants :

```
0.7 paid
1.9 unpaid
2.5 paid
4.2 unpaid
6.0 unpaid
6.5 unpaid
7.5 unpaid
8.1 unpaid
8.7 paid
10.0 paid
```

Prédire

Il n’y a vraiment pas assez de données pour établir un modèle – vous pourriez être tenté de prédire que les utilisateurs peu ou très expérimentés sont « payants », alors que les utilisateurs d’expérience moyenne sont « non payants »

```
def predict_paid_or_unpaid(years_experience):
    if years_experience < 3.0:
        return "paid"
    elif years_experience < 8.5:
        return "unpaid"
    else:
        return "paid"
```

Évidemment, nous avons complètement déduit les seuils à l’œil nu.

Extraire

Une manière simple (mais pas très excitante) de trouver les sujets les plus populaires consisterait à compter les mots :

1 Passer chaque sujet en minuscules (car différents utilisateurs ont pu utiliser ou non des majuscules).

2 Isoler les mots de chaque sujet.

3 Compter les résultats.

```
words_and_counts = Counter(word
                for user, interest in interests
                for word in interest.lower().split())
```

Extraire

Cela permet, par exemple, d’identifier facilement les mots qui ont plus d’une occurrence :

```
for word, count in words_and_counts.most_common():
    if count > 1:
        print(word, count)
```

Extraire

```
learning 3 java 3
python 3 big 3
data 3 hbase 2
regression 2 cassandra 2
statistics 2 probability 2
hadoop 2 networks 2
```

Cours accéléré de Python

Les fonctions et modules

Les fonctions

Une fonction est une règle qui s’applique à des entrées pour retourner un résultat en sortie. En Python, nous définissons les fonctions avec le mot-clé def :

Les modules

Certaines fonctions de Python ne sont pas chargées par défaut. C’est le cas de fonctions à la fois intégrées au langage et tierces à télécharger à part. Pour utiliser les secondes, vous devez importer les modules correspondants.

Une méthode consiste à importer le module proprement dit :

Les exceptions

En cas d’erreur, Python déclenche une exception. Si cette dernière n’est pas traitée, elle pro- voquera le plantage du programme. Vous pouvez gérer les exceptions avec try et except :

Types complexes de données

Les chaînes de caractères

Les listes

La liste est certainement la structure de données la plus fondamentale de Python. Une liste est simplement une collection ordonnée. Elle est assez similaire à ce que d’autres langages appellent un tableau, mais avec des fonctionnalités en plus.

Types complexes de données

Les tuples

Les tuples sont les cousins invariants des listes. Vous pouvez faire avec un tuple à peu près tout ce qu'il est possible de faire avec une liste, du moment que vous ne la modifiez pas. On décrit un tuple à l'aide de parenthèses (ou sans rien) au lieu de crochets :

Types complexes de données

Les dictionnaires

Le dictionnaire est une autre structure de données fondamentale. Elle associe des valeurs (values) à des clés (keys) et vous permet de récupérer rapidement la valeur qui correspond à une clé donnée :

Defaultdict

Imaginons que vous essayez de compter les mots dans un document. Une méthode simple consiste à créer un dictionnaire dont les clés sont les mots et les valeurs sont les nombres d’occurrences de ces mots. Tandis que vous passez sur chaque mot, vous pouvez incrémenter le compteur dès lors que le mot est déjà dans le dictionnaire. Dans la négative, vous l’y ajoutez :

Types complexes de données

Les compteurs

Un compteur (counter) transforme une séquence de valeurs en un objet de style defaultdict(int) qui fait correspondre des clés à des comptages :

Les ensembles

L’ensemble (set) est une autre structure de données utile : il permet de représenter une collec- tion d’éléments distincts.

Pour définir un ensemble, vous pouvez en énumérer les éléments entre accolades

Les structures de contrôle

La programmation orientée objet

Comme de nombreux langages, Python vous permet de définir des classes pour encapsuler des données et les fonctions associées. Nous les utiliserons parfois pour rendre le code plus propre et plus simple. Pour les expliquer, le plus simple consiste à construire un exemple commenté.

```
class CountingClicker:

    """une classe peut/doit posséder un docstring"""
```

Les générateurs et itérateurs

Le bon côté d'une liste, c'est que vous pouvez en extraire des éléments spécifiques grâce à leur index. Cependant, une liste d'un milliard d' éléments risque d'occuper une place considérable en mémoire. Or, si vous traitez ces éléments un à un (et non en lot), il n'y a aucune raison de les conserver à portée de main.

```
def generate_range(n): i=0

    while i < n:

        yield i # chaque appel à yield

        # produit une valeur du générateur i += 1
```

Les valeurs aléatoires grâce au module random :

Le module random produit en fait des nombres pseudo-aléatoires (c'est-à-dire déterministes) selon un état interne que vous pouvez initialiser avec la fonction random.seed si vous voulez des résultats reproductibles :

```
import random

random.seed(10) # pour obtenir le même résultat à chaque fois

# random.random() produit des nombres uniformément répartis entre
0 et 1

# c'est la fonction aléatoire que nous utiliserons le plus souvent

four_uniform_randoms = [random.random() for _ in range(4)]

# [0.5714025946899135,

# random.random() produit des nombres

# 0.4288890546751146,

# 0.5780913011344704,

# 0.2060982321395017]
```

Les expressions rationnelles

Les expressions rationnelles permettent de chercher du texte. Elles sont incroyablement utiles, mais aussi extraordinairement complexes et il faudrait leur consacrer un livre entier.

```
import re

re_examples = [ # tous sont vrais car

    not re.match("a", "cat"),

    # 'cat' ne commence pas par 'a'

    re.search("a", "cat"),

    # 'cat' contient un 'a'

    not re.search("c", "dog"),

    # 'dog' ne contient pas de 'c'

    3 == len(re.split("[ab]", "carbs")),

    # se coupe en a ou b en ['c','r','s']

    "R-D-" == re.sub("[0-9]", "-", "R2D2")

    # remplace les chiffres par des tirets

]

assert all(re_examples), "all the regex examples should be True"
```

zip et le déballage d’arguments

Nous aurons souvent besoin de la fonction zip pour agréger deux éléments itérables ou plus. Elle transforme ainsi plusieurs listes en une liste unique de tuples :

```
list1 = ['a', 'b', 'c']

list2 = [1, 2, 3]

# zip est paresseuse ; vous devez donc procéder plus ou moins comme suit

[pair for pair in zip(list1, list2)] # is

[('a', 1), ('b', 2), ('c', 3)]
```

Visualisation des données

Visualisation des données

1. Matplotlib
2. Les graphiques à barres
3. Les courbes
4. Les nuages de points

Algèbre linéaire

Les vecteurs

Sur un plan abstrait, les vecteurs sont des objets qui peuvent être ajoutés entre eux et/ou mul- tipliés par des scalaires (autrement dit des nombres), pour former d'autres vecteurs.

Concrètement (pour nous), les vecteurs sont des points dans un espace de dimension finie.

Par exemple, si vous gérez la taille, le poids et l'âge d'un grand nombre de personnes, vous pouvez considérer vos données comme des vecteurs à trois dimensions [taille, poids, âge].

La méthode la plus simple consiste à représenter les vecteurs par une liste de nombres.

```
from typing import List Vector =
List[float]

height_weight_age = [70, # en pouces, soit
167 centimètres

                     170, # en livres, soit
61 kilogrammes

                     40] # années

grades = [95, # examen 1

          80, # examen 2

          75, # examen 3

          62 ] # examen 4
```

Les matrices

Une matrice est une collection de nombres à deux dimensions. Nous représenterons les matrices comme des listes de listes, toutes les listes internes ayant la même taille et représentant les lignes de la matrice. Si A est une matrice, alors A[i][j] est l'élément de la i-ème ligne et de la j-ème colonne. Par convention mathématique, nous utiliserons souvent des majuscules pour représenter les matrices.

```
# un autre alias de type

Matrix = List[List[float]]

# A présente 2 lignes et 3
colonnes

A = [[1, 2, 3], [4, 5, 6]]

# B présente 3 lignes et 2
colonnes

B = [[1, 2], [3, 4], [5, 6]]
```

Statistique

Réalisé en live

Probabilités

Probabilité conditionnelle

Quand deux événements E et F sont indépendants, par définition nous avons :

$$P(E,F) = P(E)\,P(F)$$

S’ils ne sont pas nécessairement indépendants (et si la probabilité de F n’est pas 0), alors nous définissons la probabilité de E conditionnelle selon F telle que :

$$P(E|F) = P(E,F) / P(F)$$

Probabilité conditionnelle

Un exemple classique, un peu délicat, consiste à considérer une famille avec deux enfants. Si nous supposons que :

• chaque enfant a autant de chances d'être une fille que d'être un garçon ;

• le sexe du deuxième enfant est indépendant du sexe du premier ; alors l'événement aucune fille a une probabilité de 1/4, l'événement un garçon, une fille à la probabilité 1/2 et

l'événement deux filles a une probabilité de 1/4.

Vérification en live

Théorème de Bayes

Un des meilleurs amis de l'expert en data science est le théorème de Bayes. Il permet d'« inverser » les probabilités conditionnelles.

$$P(E|F) = P(E,F)/P(F)$$

$$= P(F|E)\, P(E)/P(F)$$

Théorème de Bayes

L'événement F peut être divisé en deux événements mutuellement exclusifs F et E et F et pas-E. Si nous écrivons ¬E pour pas-E (c'est-à-dire E ne se produit pas), alors on obtient la relation suivante :

$$P(F) = P(F,E) + P(F,\neg E)$$

On en déduit la formulation la plus fréquente du théorème de Bayes :

$$P(E|F) = P(F|E)\,P(E) / [P(F|E)P(E) + P(F|\neg E)P(\neg E)]$$

Variables aléatoires

Une variable aléatoire est une variable dont les valeurs possibles sont distribuées selon une loi de probabilité donnée.

Une variable aléatoire très simple correspond à 1 si une pièce lancée indique face et 0 si elle indique pile.

Une variable plus complexe, par exemple, mesurera le nombre de faces observées quand on lance la pièce 10 fois ou correspondra à une valeur prise dans une liste range(10) pour laquelle chaque nombre a la même chance de sortir.

Distributions continues

Démo live

Distribution normale

La distribution normale est la reine des distributions. C’est la classique courbe en cloche. Elle est complètement déterminée par deux paramètres : son espérance μ (mu) et son écart-type σ (sigma).

$$f(x|\mu,\sigma) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

Démo live

Théorème de la limite centrale

Une des raisons de l'intérêt porté à la distribution normale tient au théorème de la limite centrale : une variable aléatoire définie comme la moyenne d'un grand nombre de variables aléatoires indépendantes et de distribution identique suit approximativement une loi (distribution) normale.

En particulier, si x1 ... xn sont des variables aléatoires d'espérance μ et d' écart-type σ et si n est grand, alors :

$$\frac{1}{n}(x_1 + \cdots + x_n)$$

est approximativement de distribution normale avec l'espérance μ et l'écart-type

Hypothèses et inférences

Test statistique d’une d’hypothèse

Souvent, les data scientists veulent tester une hypothèse donnée pour savoir si elle a des chances de se vérifier. Pour nous, les hypothèses sont des affirmations que nous pouvons tra- duire en termes de statistiques sur les données ; des affirmations comme cette pièce n’est pas truquée ou les data scientists préfèrent Python à R

Exemple : lancer de pièce

Imaginons que nous voulons vérifier qu’une pièce n’est pas truquée. Émettons l’hypothèse qu’elle a une probabilité p de donner face. Notre hypothèse nulle stipule que la pièce n’est pas truquée ; en d’autres termes, $p = 0,5$. Nous allons tester cette hypothèse contre l’autre hypothèse : $p \neq 0,5$.

Test statistique d’une d’hypothèse

Notre test suppose donc de lancer la pièce n fois et de compter le nombre X de sorties face. Chaque lancer de pièce est un essai de Bernoulli, ce qui veut dire que X est une variable aléatoire Binomiale(n, p), que nous pouvons approcher par une distribution normale

```python
from typing import Tuple

import math

def normal_approximation_to_binomial(n: int, p: float)
-> Tuple[float, float]:

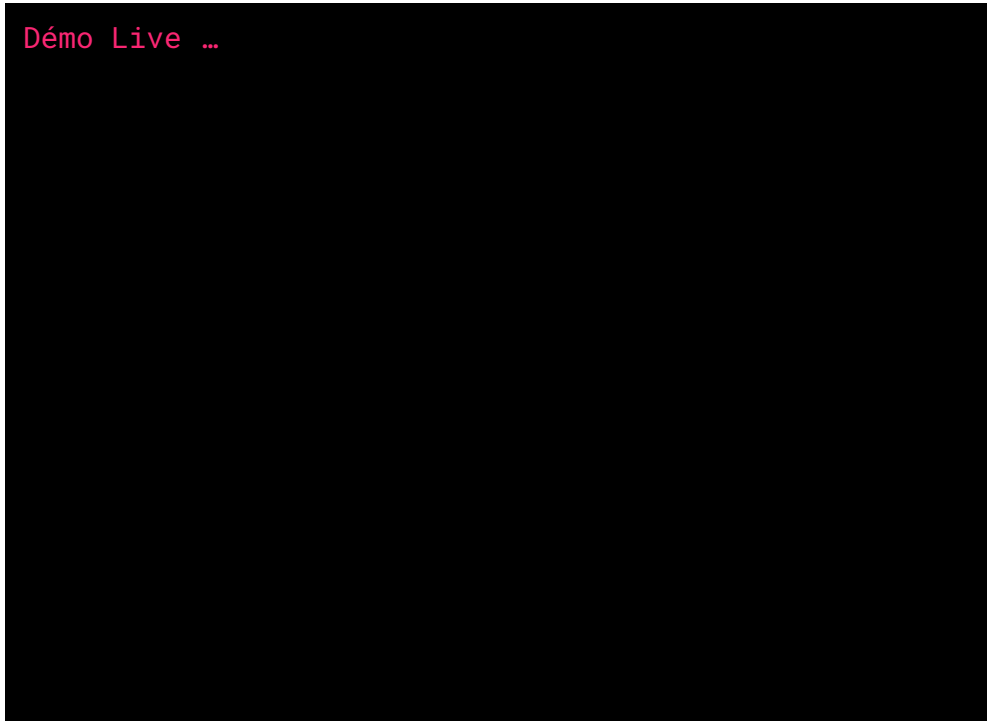
    """trouver mu et sigma correspondant à
    Binomiale(n,p)"""

    mu = p * n

    sigma = math.sqrt(p * (1 - p) * n)
    return mu, sigma
```

Test statistique d’une d’hypothèse

Chaque fois qu’une variable aléatoire suit une distribution normale, nous pouvons utiliser *normal_cdf* pour prévoir la probabilité que la valeur réalisée s’inscrive à l’intérieur (ou à l’extérieur) d’un intervalle donné :



Test statistique d’une d’hypothèse

Nous pouvons aussi faire le raisonnement inverse : rechercher la zone excluant les extrémités de la distribution ou l’intervalle (symétrique) autour de l’espérance qui correspond à un certain niveau de vraisemblance.

Par exemple, si nous voulons trouver un intervalle centré sur l’espérance et contenant 60 % de probabilité, nous devons rechercher les seuils à proximité desquels les extrémités inférieures et supérieures contiennent chacune 20 % de la probabilité :

Démo Live …

En particulier, supposons que nous lançons la pièce n = 1 000 fois.

Si notre hypothèse de non-trucage est vraie, X devrait être distribuée approximativement selon une courbe normale d’espérance 500 et d’écart-type 15,8

```
mu_0, sigma_0 = normal_approximation_to_binomial(1000, 0.5)
```

Test statistique d’une d’hypothèse

Nous devons prendre une décision sur le sens que nous donnons à « significatif » afin de valider ou d’infirmer notre hypothèse : jusqu’à quel point sommes-nous prêts à accepter une erreur de type 1 (faux positif) qui nous amènerait à rejeter H0 même si elle est vraie ?

Pour des raisons qui se perdent dans la nuit des temps, ce seuil est souvent valorisé à 5 % ou 1 %.

Choisissons 5 %.

```
lower_bound, upper_bound =
normal_two_sided_bounds(0.95, mu_0, sigma_0)
```

Test statistique d’une d’hypothèse

```
# (469, 531)
lower_bound, upper_bound = normal_two_sided_bounds(0.95, mu_0, sigma_0)
```

En supposant que p soit réellement égale à 0,5 (c’est-à-dire que H0 est vraie), il y a seulement 5 % de risque que nous observions un X qui se situe en dehors de cet intervalle, ce qui est exac- tement la valeur significative voulue. Autrement dit, si H0 est vraie, alors ce test va donner le résultat correct environ 19 fois sur 20.

Souvent, nous nous intéressons à la puissance d'un test, c'est-à-dire à la probabilité de ne pas faire une erreur de type 2 (faux négatif), autrement dit de ne pas rejeter H0 bien qu'elle soit fausse.

Pour mesurer ce résultat, nous devons d'abord préciser ce que signifie exactement H0 est fausse. Savoir seulement que p n'est pas égale à 0,5 ne vous renseigne pas vraiment sur la distribution de X.

En particulier, examinons ce qui arrive si p est réellement égale à 0,55, ce qui veut dire que la pièce est légèrement biaisée en faveur de face. Dans ce cas, nous pouvons calculer la puissance du test comme suit :

```
# les limites à 95 % sur la base de l'hypothèse que p vaut 0.5
lo, hi = normal_two_sided_bounds(0.95, mu_0, sigma_0)
# mu et sigma sur la base de p = 0.55
mu_1, sigma_1 = normal_approximation_to_binomial(1000, 0.55)
# Une erreur de type 2 signifie que nous n'avons pas réussi à rejeter
# l'hypothèse nulle ce qui se produira si X est encore dans notre
# intervalle d'origine.
type_2_probability = normal_probability_between(lo, hi, mu_1, sigma_1)
power = 1 - type_2_probability # 0.887
```

Imaginons à présent que l'hypothèse nulle correspond au fait que la pièce n'est pas biaisée en faveur de face, ou que p ≤ 0,5.

Dans ce cas, nous voulons qu'un test unilatéral rejette l'hypo- thèse nulle quand X est beaucoup plus grand que 500, mais non quand X est plus petit que 500.

De fait, tester une valeur significative de 5 % revient à dire utiliser normal_probability_below pour trouver les seuils en dessous desquels se situent 95 % des probabilités :

```
hi = normal_upper_bound(0.95, mu_0, sigma_0)
# Vaut 526 (<531, car nous avons besoin d'une probabilité plus grande
# dans la partie supérieure).
type_2_probability = normal_probability_below(hi, mu_1, sigma_1)
power = 1 - type_2_probability # 0.936
```

p-values

Une autre manière d'envisager le test précédent consiste à utiliser les p-values. Au lieu de choisir des limites basées sur un seuil de probabilités, nous calculons la probabilité (en supposant que H0 est vraie) de voir apparaître une valeur au moins aussi extrême que celle que nous avons véritablement observée.

```
def two_sided_p_value(x: float, mu: float
= 0, sigma: float = 1) -> float:

    """ Quelle est la probabilité de voir
apparaître une valeur

    au moins aussi extrême que x (dans
une direction ou l'autre)

    si nos valeurs sont issues d'un N(mu,
sigma) ? """
```

```
    if x >= mu:

        # x est plus grand que l'espérance

        # la queue de distribution est donc tout ce qui
est supérieur à x

        return 2 * normal_probability_above(x, mu, sigma)

    else:

        # x est plus petit que l'espérance

        # la queue de distribution est donc tout ce qui
est inférieur à x

        return 2 * normal_probability_below(x, mu, sigma)
```

p-values

Si nous avions vu 530 fois face, nous ferions le calcul suivant :

```
two_sided_p_value(529.5, mu_0, sigma_0) # 0.062
```

Pour vous convaincre que c’est une estimation raisonnable, faisons une simulation :

Comme la p-value est supérieure à notre valeur significative de 5 %, nous ne rejetons pas l’hypothèse nulle.

```
import random extreme_value_count = 0 for _ in
range(1000):

num_heads = sum(1 if random.random() < 0.5 else 0 #
compteur de faces for _ in range(1000))

# en 1 000 lancers

if num_heads >= 530 or num_heads <= 470: # et
compte le nombre de fois extreme_value_count += 1

# le tirage est extrême

# la valeur p-value était de 0.062 => ~62 valeurs
extrêmes sur 1 000

assert 59 < extreme_value_count < 65,
f"{extreme_value_count}"
```

En revanche, si nous avions pris 532, la p-value serait alors :

Ce qui est inférieur à 5 % et impliquerait que nous rejetions l’hypothèse nulle. C’est exactement le même test qu’auparavant, mais avec une approche différente des statistiques.

```
two_sided_p_value(531.5, mu_0, sigma_0)
# 0.0463
```

Avertissement

Assurez-vous que vos données suivent approximativement une loi normale avant d'utiliser normal_probability_above pour calculer les p-values.

Il existe différents tests statistiques pour vérifier la normalité. Un bon début consiste à commencer par une simple représentation graphique des données.

Intervalle de confiance

Nous avons testé des hypothèses sur la probabilité p de voir sortir face, ce qui constitue un paramètre de la distribution inconnue de face. Dans ce genre de cas, il existe une troisième approche qui consiste à construire un intervalle de confiance autour de la valeur observée pour le paramètre.

Par exemple, on peut estimer la probabilité d'une pièce truquée en regardant la valeur moyenne des variables de Bernoulli qui correspondent à chaque lancer : 1 si face, 0 si pile. Si nous observons face 525 fois pour 1 000 lancers, nous estimerons que p vaut 0,525.

Quelle confiance pouvons-nous accorder à cette estimation ?

En fait, si nous connaissions la valeur exacte de p, le théorème de la limite centrale (chapitre 6) nous dit que, en moyenne, ces variables de Bernoulli devraient approximativement suivre une loi normale d'espérance p et d'écart-type `math.sqrt(p * (1 - p) / 1 000)`.

Intervalle de confiance

Ici cependant, nous ne connaissons pas p, donc nous utilisons une valeur approchée :

```
p_hat = 525 / 1000
mu = p_hat
sigma = math.sqrt(p_hat * (1 - p_hat) / 1000) # 0.0158
```

Avec l'approximation de la loi normale, nous pouvons conclure que nous sommes « confiants à 95 % » que l'intervalle suivant contiendra le vrai paramètre p :

```
normal_two_sided_bounds(0.95, mu, sigma) # [0.4940, 0.5560]
```

Remarque

Cette affirmation concerne l'intervalle, mais non p. Vous devez la comprendre comme l'affirmation que, si vous répétiez l'expérience plusieurs fois, 95 % du temps la « vraie » valeur (qui est la même à chaque fois) se situerait dans l'intervalle de confiance observé (qui peut être différent à chaque fois).

Intervalle de confiance

Plus particulièrement, nous ne concluons pas que notre pièce est truquée, car 0,5 tombe dans notre intervalle de confiance. Si nous avions observé face 540 fois, nous aurions :

```
p_hat = 540 / 1000
mu = p_hat
sigma = math.sqrt(p_hat * (1 - p_hat) / 1000) # 0.0158
normal_two_sided_bounds(0.95, mu, sigma) # [0.5091, 0.5709]
```

Dans ce cas, la « pièce non truquée » ne se situe pas dans l'intervalle de confiance. L'hypothèse « pièce non truquée » ne réussit pas le test qu'elle était censée réussir dans 95 % des cas si elle était vrai

P-hacking

Par définition, une procédure qui rejette à tort l'hypothèse nulle seulement 5 % du temps rejette l'hypothèse nulle à tort dans 5 % des cas :

```
from typing import List

def run_experiment() -> List[bool]:
    """lancer une pièce non truquée 1 000 fois, Vrai = face, Faux = pile"""
    return [random.random() < 0.5 for _ in range(1000)]

def reject_fairness(experiment: List[bool]) -> bool:
    """utilisation des niveaux de signification de 5 %"""
    num_heads = len([flip for flip in experiment if flip])
    return num_heads < 469 or num_heads > 531
```

Cela veut dire que, si vous essayez de trouver des résultats « significatifs », vous le pourrez en général. Testez suffisamment d'hypothèses vis-à-vis de votre jeu de données et une d'entre elles apparaîtra certainement significative. Éliminez les extrêmes adéquats et vous pourrez probablement faire passer votre valeur p sous la barre des 0,05.

Exemple : effectuer un test A/B

Une de vos principales missions chez DataSciencester est « d'optimiser l'expérience client », ce qui est un euphémisme signifiant que vous devez inciter les internautes à cliquer sur les annonces publicitaires.

Un de vos annonceurs a développé une nouvelle boisson énergisante qui cible les data scientists et le responsable Publicité veut que vous l'aidiez à choisir entre l'annonce

A Quel régal ! et l'annonce **B Moins de biais**.

En bon scientifique (des données), vous décidez de lancer une expérience en présentant aux visiteurs du site une des deux annonces au hasard et en relevant le nombre de personnes qui cliquent dans chaque cas.

Exemple : effectuer un test A/B

Si 990 visiteurs sur 1 000 ayant vu A cliquent sur cette annonce

alors que seulement 10 sur 1 000 de ceux qui ont vu l'annonce B cliquent sur B,

vous pouvez alors être sûr que A est meilleure que B.

Toutefois, que conclure si les différences ne sont pas si marquées ?

C'est là que nous allons faire intervenir l'inférence statistique.

Exemple : effectuer un test A/B

Soit *NA* le nombre de personnes qui ont vu l'annonce A et *nA* le nombre de celles qui ont cliqué dessus.

Nous pouvons envisager chaque visualisation d'annonce comme un essai de Bernoulli dans lequel *pA* est la probabilité que quelqu'un clique sur A.

Ensuite, (si *NA* est grand, ce qui est le cas), nous savons que *nA / NA* suit approximativement une loi normale d'espérance pA et d'écart-type

$$\sigma_A = \sqrt{\frac{p_A(1-p_A)}{N_A}}$$

De même pour l'autre annonce, *nB / NB* est approximativement une variable aléatoire normale d'espérance *pB* et d' écart-type

$$\sigma_B = \sqrt{\frac{p_B(1-p_B)}{N_B}}$$

Exemple : effectuer un test A/B

On exprime ça

```
def estimated_parameters(N: int, n: int) ->
Tuple[float, float]: p=n/N

     sigma = math.sqrt(p * (1 - p) / N)

     return p, sigma
```

Si nous considérons que les deux lois normales sont indépendantes (ce qui paraît raisonnable, car les essais de Bernoulli individuels doivent être indépendants), alors leur différence devrait également suivre une loi normale, d'espérance *pB - pA* et d'écart-type

$$\sqrt{\sigma_A^2 + \sigma_B^2}$$

Exemple : effectuer un test A/B

Cela veut dire que nous pouvons tester l'hypothèse nulle selon laquelle *pA* et *pB* sont les mêmes (c'est-à-dire *pA – pB = 0*) en utilisant la statistique suivante :

```
def a_b_test_statistic(N_A: int, n_A: int, N_B: int, n_B: int) -> float:
    p_A, sigma_A = estimated_parameters(N_A, n_A)
    p_B, sigma_B = estimated_parameters(N_B, n_B)
    return (p_B - p_A) / math.sqrt(sigma_A ** 2 + sigma_B ** 2)
```

Cela devrait correspondre approximativement à une distribution normale standard.

Exemple : effectuer un test A/B

Par exemple, si le slogan A récolte 200 clics pour 1 000 vues, alors que le slogan B en récolte 180, la statistique vaut ainsi :

```
z = a_b_test_statistic(1000, 200, 1000, 180) # -1.14
```

La probabilité de voir une si grande différence si les espérances étaient égales est de :

```
two_sided_p_value(z) # 0.254
```

C’est suffisant pour vous empêcher de conclure qu’il y a une grande différence.

Exemple : effectuer un test A/B

D’un autre côté, si le slogan B reçoit seulement 150 clics, alors :

```
z = a_b_test_statistic(1000, 200, 1000, 150) # -2.95
two_sided_p_value(z) # 0.003
```

Cela signifie qu’il y a seulement une probabilité de 0,003 de voir une grande différence dans le cas où les annonces ont la même efficacité.

C’est suffisant pour vous empêcher de conclure qu’il y a une grande différence.

Descente de gradient

Idée cachée derrière la descente de gradient

Soit une fonction f qui accepte en entrée un vecteur de nombres réels et qui donne en sortie un unique nombre réel. Rappelez-vous la fonction *sum_of_squares*

Nous aurons souvent à maximiser (ou minimiser) ce genre de fonction. En d'autres termes,

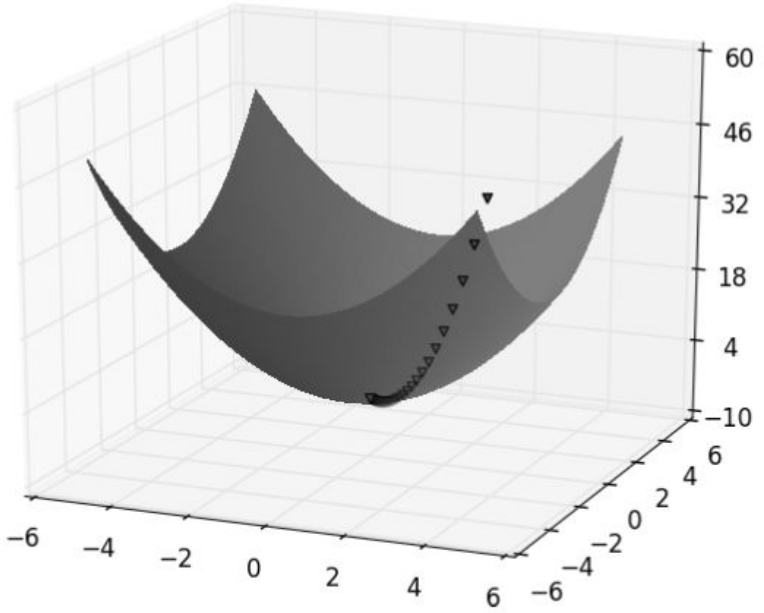
nous devrons trouver l'entrée v qui produit la plus grande (ou la plus petite) valeur possible.

Pour des fonctions comme les nôtres, le gradient (si vous avez oublié vos cours de calcul infinitésimal, c'est le vecteur des dérivées partielles) donne la direction vers laquelle la fonction augmente le plus rapidement.

Idée cachée derrière la descente de gradient

Une méthode pour maximiser une fonction consiste à

1. prendre un point de départ au hasard,
2. calculer le gradient,
3. appliquer un petit déplacement dans la direction de ce dernier (c'est-à-dire la direction qui correspond à l'augmentation la plus importante),
4. puis répéter le processus à partir de ce nouveau point de départ.



Remarque

Si une fonction possède un unique minimum global, cette procédure le trouvera certainement. Si une fonction possède plusieurs minima locaux, la procédure pourrait conduire à en désigner un qui n'est pas le minimum global ; dans ce cas, il faudra recommencer à partir de différents points de départ. Si une fonction ne possède pas de minimum, la procédure peut se poursuivre sans fin.

Estimer le gradient

Si *f(x)* est une fonction à une variable, sa dérivée au point *x* mesure

comment *f(x)* évolue

quand on effectue un petit changement *h* sur *x*.

La dérivée est définie comme la limite des quotients de différence quand *h* tend vers zéro.

```
from typing import Callable

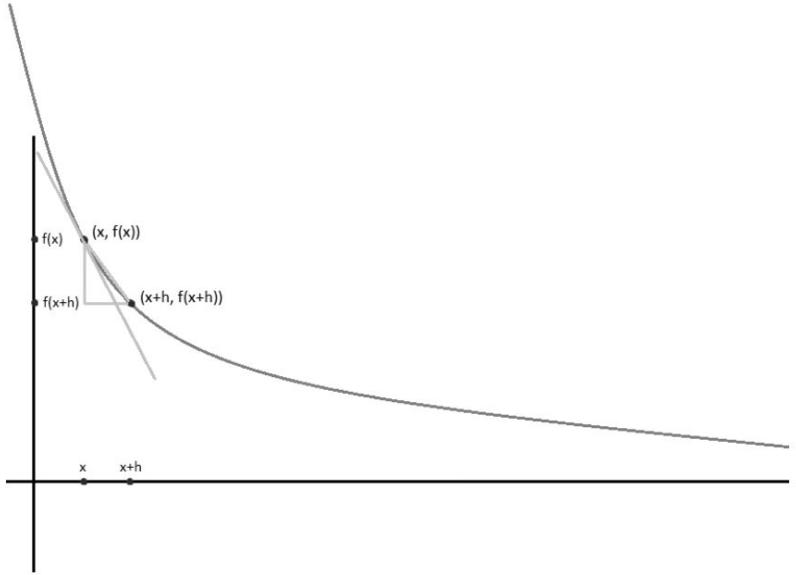
def difference_quotient(f: Callable[[float], float], x: float, h: float) -> float:
    return (f(x + h) - f(x)) / h
```

Estimer le gradient

La ***dérivée*** est la pente de la tangente en (x,f(x)),

alors que ***le quotient de différence*** représente la pente de la ligne presque-tangente qui passe par (x + h, f(x + h)).

Quand h devient de plus en plus petit, la presque-tangente s'approche de plus en plus de la tangente



Pour de nombreuses fonctions, il est facile de calculer les dérivées exactes. f(x) = x^2 => f'(x) = 2x

Estimer le gradient

Quand une fonction f présente plusieurs variables,

elle a plusieurs dérivées partielles,

chacune indiquant comment f évolue quand nous procédons à de petites modifications sur une seule des variables d’entrée.

Estimer le gradient

Nous calculons sa i-ème dérivée en la traitant comme une fonction à une seule variable de rang i, tandis que les autres sont fixes :

```
def partial_difference_quotient(f: Callable[[Vector], float],
                                v: Vector,
                                i: int,
                                h: float) -> float:
    """calcule le i-ème quotient partiel des différences de f en v"""
    # ajoute h pour ajuster le i-ème élément de v
    w = [v_j + (h if j == i else 0) # ajoute h pour ajuster l'élément i de
v
    for j, v_j in enumerate(v)]
    return (f(w) - f(v)) / h
```

Estimer le gradient

Cela permet ensuite d'estimer le gradient de la même façon :

```
def estimate_gradient(f: Callable[[Vector], float],
                      v: Vector,
                      h: float = 0.0001):
    return [partial_difference_quotient(f, v, i, h) for i in range(len(v))]
```

Il est facile de vérifier que la fonction sum_of_squares est de valeur minimale quand son entrée v est un vecteur de zéros,

mais imaginons que vous l'ignoriez.

Utilisons le gradient **pour trouver le minimum dans le cas de vecteurs à trois dimensions**.

1. Nous prendrons un point de départ au hasard
2. et nous effectuerons de très petits déplacements dans la direction opposée au gradient,
3. jusqu'à ce que nous atteignions un point où le gradient est très faible

```python
import random
from scratch.linear_algebra import distance, add, scalar_multiply

def gradient_step(v: Vector, gradient: Vector, step_size: float) -> Vector:
"""se déplacer de step_size dans la direction de gradient depuis v"""
    assert len(v) == len(gradient)
    step = scalar_multiply(step_size, gradient)
    return add(v, step)

def sum_of_squares_gradient(v: Vector) -> Vector:
    return [2 * v_i for v_i in v]


# choisir un point de départ aléatoire
v = [random.uniform(-10, 10) for i in range(3)]

for epoch in range(1000):
    grad = sum_of_squares_gradient(v) # calculer le gradient en v
    v = gradient_step(v, grad, -0.01) # effectuer un pas de gradient négatif
    print(epoch, v)


assert distance(v, [0, 0, 0]) < 0.001 # v devrait être proche de 0
```

S’adapter à des modèles

Dans le cas habituel, nous disposons d’un ensemble de données et d’un modèle (hypothétique). Ce dernier dépend (de manière différentiable) d’un ou plusieurs paramètres.

Nous disposons en outre d’une fonction de perte qui mesure la qualité d’adaptation du modèle à nos données (le plus petit sera le mieux)

nous pouvons utiliser une descente de gradient pour trouver des paramètres de modèle qui atténuent la perte autant que faire se peut

```
# x va de -50 à 49 ; y vaut toujours 20 * x + 5
inputs = [(x, 20 * x + 5) for x in range(-50, 50)]
```

Dans ce cas, nous connaissons les paramètres de la relation linéaire entre `x` et `y`.

S’adapter à des modèles

Imaginons à présent que nous souhaitions les identifier à partir des données. Nous utilisons alors une descente de gradient pour identifier la pente et l’ordonnée à l’origine qui minimisent l’erreur quadratique moyenne.

Nous commencerons avec une fonction qui détermine le gradient selon l’erreur issue d’un seul point de données

```
def linear_gradient(x: float, y: float, theta: Vector) -> Vector:
    slope, intercept = theta
    predicted = slope * x + intercept # prévision du modèle
    error = (predicted - y) # l’erreur est (prévue - réelle)
    squared_error = error ** 2 # nous minimisons l’erreur quadratique
    grad = [2 * error * x, 2 * error] # en utilisant son gradient
    return grad
```

Réfléchissons à la signification du gradient.

Imaginons que, pour une valeur x, notre prévision soit trop grande.

S’adapter à des modèles

Pour un ensemble complet de données, nous examinerons l’erreur quadratique moyenne. Et le gradient de l’erreur quadratique moyenne est simplement la moyenne des gradients pris individuellemen

Voici comment nous allons procéder.

1. Commencer avec une valeur aléatoire pour thêta.
2. Calculer la moyenne des gradients.
3. Ajuster thêta dans cette direction.
4. Répéter.

S’adapter à des modèles

Après un grand nombre d’epochs (d’étapes), qui désignent les différentes passes sur l’ensemble de données), nous devrions obtenir quelques éléments d’information ressemblant aux paramètres appropriés

```
from scratch.linear_algebra import vector_mean

# commencer avec des valeurs aléatoires pour la pente et y0
theta = [random.uniform(-1, 1), random.uniform(-1, 1)]
learning_rate = 0.001

for epoch in range(5000):
    # calculer la moyenne des gradients
    grad = moyenne_vecteurs([gradient_linéaire(x, y, thêta)
    for x, y in inputs])
    # faire un pas dans cette direction
    theta = gradient_step(theta, grad, -learning_rate)
    print(epoch, theta)
    slope, intercept = theta

assert 19.9 < slope < 20.1, "la pente devrait être d’environ 20"
assert 4.9 < intercept < 5.1, "l’interception devrait être d’environ 5"
```

Mini-lot et descente du gradient stochastique

L’inconvénient avec la précédente approche tient à ce qu’il faut évaluer les gradients sur tout l’ensemble des données avant de pouvoir adopter un pas de gradient et d’actualiser nos paramètres.

Dans le cas présent, tout allait bien car notre ensemble de données n’était constitué que de 100 paires. Le calcul du gradient réclamait donc peu de ressources.

En revanche, vos (vrais) modèles présenteront souvent des ensembles de données de grande envergure et des calculs de gradient plus gourmands en ressources. Dans de tels cas, vous devrez plus souvent faire appel à différents pas de gradient

Mini-lot et descente du gradient stochastique

Pour ce faire, nous utilisons une technique appelée descente de gradient par mini-lots. Dans celle-ci, nous calculons le gradient (et adoptons un pas de gradient) en nous fondant sur un « mini-lot » échantillonné à partir d'un ensemble de données plus volumineux

Nous commencerons avec une fonction qui détermine le gradient selon l'erreur issue d'un seul point de données

```
from typing import TypeVar, List, Iterator

T = TypeVar('T') # ceci nous permet de typer des fonctions " génériques "

def minibatches(dataset: List[T],
                batch_size: int,
                shuffle: bool = True) -> Iterator[List[T]]:
    """génère des mini-lots à partir de l'ensemble de données"""

    # débuter des index 0, batch_size, 2 * batch_size, …
    batch_starts = [start for start in range(0, len(dataset), batch_size)]
    if shuffle: random.shuffle(batch_starts) # mélange les lots
    for start in batch_starts:
        end = start + batch_size
        yield dataset[start:end]
```

Mini-lot et descente du gradient stochastique

Nous pouvons à présent résoudre notre problème en utilisant des mini-lots :

```
theta = [random.uniform(-1, 1), random.uniform(-1, 1)]

for epoch in range(1000):
    for batch in minibatches(inputs, batch_size=20):
        grad = vector_mean([linear_gradient(x, y, theta)
                            for x, y in batch])
        theta = gradient_step(theta, grad, -learning_rate)
        print(epoch, theta)
        slope, intercept = theta

assert 19.9 < slope < 20.1, "la pente devrait être d'environ 20"
assert 4.9 < intercept < 5.1, "l'interception devrait être d'environ 5"
```

Mini-lot et descente du gradient stochastique

Dans une variante appelée la descente du gradient stochastique, vous pouvez adopter des pas de gradient en fonction d'un exemple d'entraînement à la fois

```
theta = [random.uniform(-1, 1), random.uniform(-1, 1)]

for epoch in range(100):
    for x, y in inputs:
        grad = linear_gradient(x, y, theta)
        theta = gradient_step(theta, grad, -learning_rate)
        print(epoch, theta)
        slope, intercept = theta

assert 19.9 < slope < 20.1, "la pente devrait être d'environ 20"
assert 4.9 < intercept < 5.1, "l'interception devrait être d'environ 5"
```

Collecte des données

pandas

est probablement l’outil Python de base

pour nettoyer, transformer, manipuler et exploiter les données.

stdin et stdout

Lecture de fichiers

Web-scrapping

Utilisation des API

JSON et XML

Travail sur les données

Explorer les données

Après avoir identifié les questions auxquelles vous cherchez des réponses et mis la main sur quelques données,

la tentation est grande d'entrer directement dans le vif du sujet et de construire des modèles pour en tirer des réponses.

Vous devez résister à la tentation.

Votre première étape doit consister à **explorer vos données**

Données à une dimension

Le cas le plus simple correspond à un ensemble de données à une seule dimension, à savoir, en fait, une collection de nombres.

Par exemple, ce peut être

- la durée moyenne passée chaque jour sur le site,
- le nombre de fois qu'un tutoriel sur la data science a été vu parmi une collection de vidéos,
- ou encore le nombre de pages que compte chaque livre consacré à la data science dans votre bibliothèque.

Données à une dimension

La première étape consiste évidemment à réaliser quelques statistiques de synthèse.

Vous voulez savoir

- de combien de données élémentaires (points de données) vous disposez,
- quelle est la plus petite,
- la plus grande,
- l’espérance et quel est
- l’écart-type.

Données à une dimension

Cependant, tout ceci ne vous aidera pas forcément à comprendre vos données. L'étape suivante passera donc avantageusement par la création d'un histogramme ; vous devrez répartir les données en tranches distinctes et compter combien figurent dans chaque tranche

```
from typing import List, Dict
from collections import Counter
import math
import matplotlib.pyplot as plt

def bucketize(point: float, bucket_size: float) -> float:
    """dirige le point vers le prochain plus petit multiple de
bucket_size"""
    return bucket_size * math.floor(point / bucket_size)

def make_histogram(points: List[float], bucket_size: float) ->
Dict[float, int]:
    """sépare les points et les compte dans chaque tranche"""
    return Counter(bucketize(point, bucket_size) for point in points)

def plot_histogram(points: List[float], bucket_size: float, title: str =
""):
    histogram = make_histogram(points, bucket_size)
    plt.bar(histogram.keys(), histogram.values(), width=bucket_size)
    plt.title(title)
```

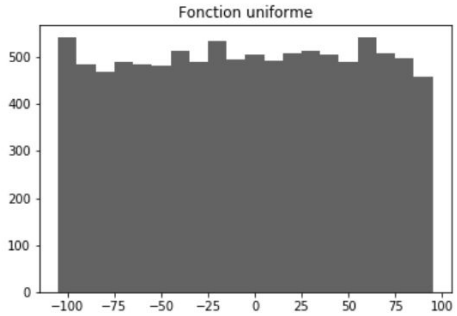
Données à deux dimensions

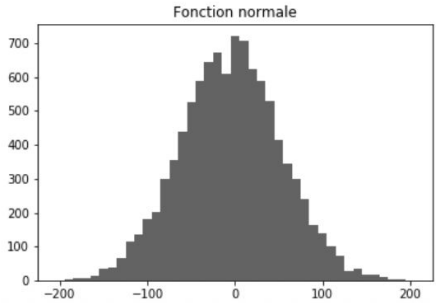
Prenons par exemple les deux jeux de données suivants :

```
import random
from scratch.probability import inverse_normal_cdf
random.seed(0)

# distribution uniforme entre -100 et 100
uniform = [200 * random.random() - 100 for _ in range(10000)]

# distribution normale avec espérance 0 et écart-type 57
normal = [57 * inverse_normal_cdf(random.random())
              for _ in range(10000)]
```





Dans les deux cas, l'espérance est proche de 0 et l'écart-type de 57. Pourtant, les deux distributions sont très différentes.

Données à deux dimensions

Imaginons maintenant que votre jeu de données présente deux dimensions.

Par exemple, en plus de la durée quotidienne en minutes de présence sur le site,

vous connaissez le nombre d'années d'expérience par utilisateur. Bien sûr, vous voulez comprendre chaque dimension individuellement.

Néanmoins, il est probable que vous vouliez surtout examiner la dispersion des données

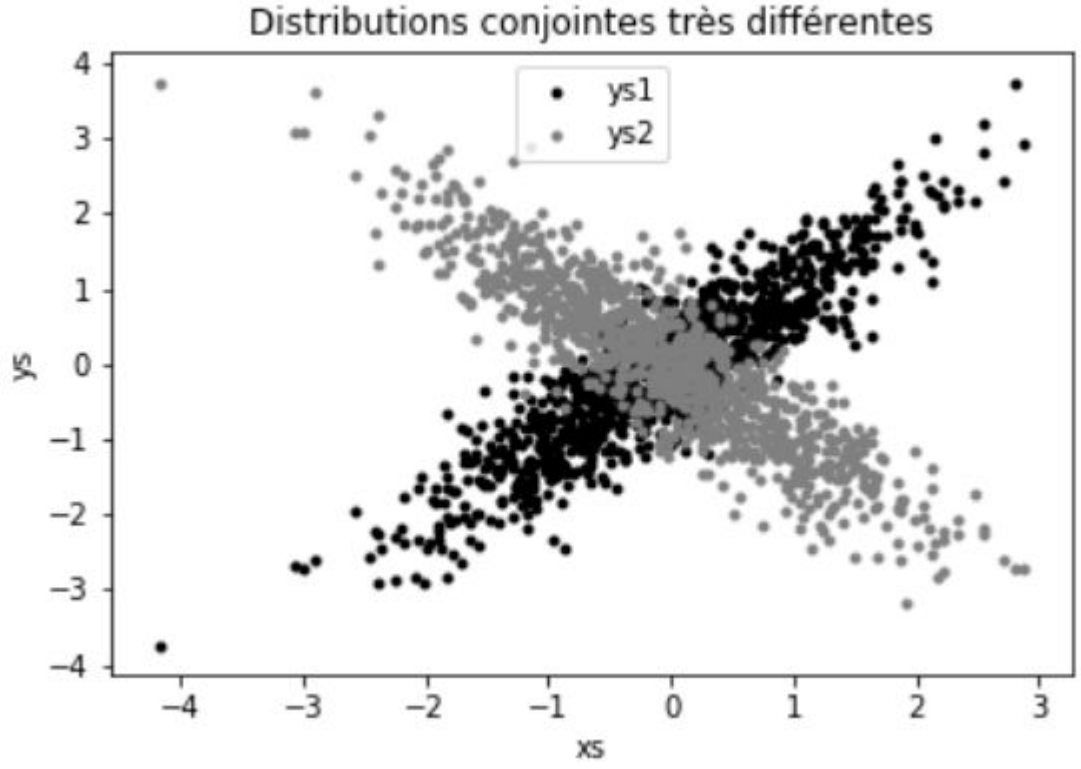
```
def random_normal() -> float:
    """retourne une distribution normale de paramètres aléatoires"""
    return inverse_normal_cdf(random.random())

xs = [random_normal() for _ in range(1000)]
ys1 = [ x + random_normal() / 2 for x in xs]
ys2 = [-x + random_normal() / 2 for x in xs]
```

Données à deux dimensions

Si vous deviez exécuter plot_histogram sur ys1 et ys2, vous obtiendriez des graphiques très proches (en fait, les deux suivent une distribution normale de même espérance et de même écart-type).

```
plt.scatter(xs, ys1, marker='.', color='black', label='ys1')
plt.scatter(xs, ys2, marker='.', color='gray', label='ys2')
plt.xlabel('xs')
plt.ylabel('ys')
plt.legend(loc=9)
plt.title("Distributions conjointes très différentes")
plt.show()
```


Distributions conjointes très différentes
ys1
ys2
ys
xs
4
3
2
1
0
−1
−2
−3
−4
−4
−3
−2
−1
0
1
2
3

Données à plusieurs dimensions

la question est de savoir comment celles-ci sont reliées entre elles.

Une méthode simple consiste à **examiner la matrice de corrélation**

dans laquelle **la donnée à l'intersection de la ligne i et de la colonne j** est **la corrélation entre la i-ème et la j-ème dimension des données**

```
from scratch.linear_algebra import Matrix, Vector, make_matrix

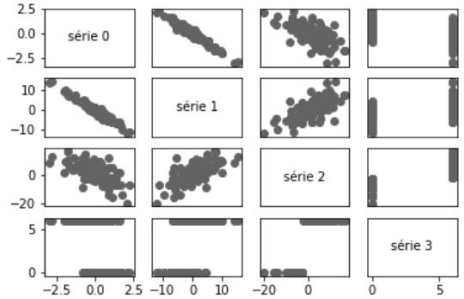
def correlation_matrix(data: List[Vector]) -> Matrix:
"""Retourne la matrice carrée dont l'entrée de rang (i,j) est la
corrélation entre les colonnes i et j des données."""

    def correlation_ij(i: int, j: int) -> float:
        return correlation(data[i], data[j])

    return make_matrix(len(data), len(data), correlation_ij)
```

Exercice

Exercice: Pour une approche plus visuelle (si vous avez peu de dimensions), construisez une matrice de nuages de points pour illustrer les nuages correspondant aux paires



Pour ce faire, nous utilisons *plt.subplots* qui nous permet de créer des sous-fenêtres graphiques dans notre schéma.

Utiliser NamedTuple

Dataclass

Le recours aux dict constitue une manière courante de représenter des données :

```
import datetime
stock_price = {'closing_price': 102.06,
    'date': datetime.date(2014, 8, 29),
    'symbol': 'AAPL'}
```

Pourtant, cette solution est loin d'être idéale et ce pour plusieurs raisons. Cette représentation est quelque peu inefficace (un dict induit un certain coût). Ainsi, si le nombre de vos cours d'action est important, ceux-ci occuperont davantage de mémoire que nécessaire. Globalement, cet aspect a peu d'importance.

Plus important est l'accès aux éléments via une clé de dict, car cette technique favorise les erreurs. Le code suivant s'exécute sans erreur et pourtant il est incorrect.

```
# oups ! Erreur typographique
stock_price['cosing_price'] = 103.06
```

Dataclass

Enfin, si nous pouvons annoter les dictionnaires uniformes par type :

```
prices: Dict[datetime.date, float] = {}
```

Il n'existe aucun moyen secourable d'annoter des dictionnaires-en-tant-que-données **qui présentent un grand nombre de types de valeur différents**. Nous perdons ainsi la puissance des indications fournies via les types.

Comme solution de substitution, Python inclut une classe namedtuple qui se comporte comme un tuple mais avec des emplacements nommés

```
from collections import namedtuple

StockPrice = namedtuple('StockPrice', ['symbol', 'date', 'closing_price'])
price = StockPrice('MSFT', datetime.date(2018, 12, 14), 106.03)
assert price.symbol == 'MSFT'
assert price.closing_price == 106.03
```

Données à plusieurs dimensions

Remarquez que nous n’avons toujours pas résolu la question de l’annotation par type.

Nous la résolvons en utilisant la variante typée, NamedTuple

```
from typing import NamedTuple
class StockPrice(NamedTuple):
    symbol: str
    date: datetime.date
    closing_price: float

    def is_high_tech(self) -> bool:
        """c’est une classe ; nous pouvons donc aussi ajouter des méthodes"""
        return self.symbol in ['MSFT', 'GOOG', 'FB', 'AMZN', 'AAPL']

price = StockPrice('MSFT', datetime.date(2018, 12, 14), 106.03)
assert price.symbol == 'MSFT'
assert price.closing_price == 106.03
assert price.is_high_tech()
```

Nettoyer et transformer

Dans le monde réel, les données ne sont pas propres. Le plus souvent, un peu de travail est nécessaire avant de pouvoir les utiliser. Nous en avons vu des exemples au chapitre précédent.

Exemple : il convient de convertir les chaînes de caractères en numériques à virgule flottante ou en entiers, avant de pouvoir les utiliser

Manipuler les données

compétence primordiale pour un expert en data science.

Supposons que nous travaillons sur des dict de cours d'actions :

```
data = [
    StockPrice(symbol='MSFT',
    date=datetime.date(2018, 12, 24),
    closing_price=106.03),
    # ...
]
```

Interrogeons-nous sur ces données.

Nous essayerons de repérer des motifs qui se répètent et nous prendrons du recul par rapport aux outils pour faciliter la manipulation.

Manipuler les données

Par exemple, supposons que nous cherchions le cours de clôture le plus élevé pour AAPL.

Décomposons la recherche en étapes simples

1. se limiter aux lignes AAPL ;
2. récupérer le cours de clôture closing_price de chaque ligne ;
3. trouver le maximum de ces cours.

Nous pouvons réaliser ces trois étapes d'un seul coup à l'aide d'une liste en compréhension :

```
max_aapl_price = max(stock_price.closing_price
                     for stock_price in data
                         if stock_price.symbol == "AAPL")
```

Manipuler les données

Plus généralement, nous pouvons chercher le cours de clôture le plus élevé pour chaque action de notre jeu de données. Voici une manière d'y parvenir :

1. créer un dict pour suivre les valeurs les plus élevées (nous utiliserons un *defaultdict* qui retourne –inf (moins infini) pour les valeurs manquantes, car toutes les valeurs seront forcément supérieures) ;
2. itérer sur nos données, en les actualisant.

```
from collections import defaultdict

max_prices: Dict[str, float] = defaultdict(lambda:
float('-inf'))

for sp in data:
    symbol, closing_price = sp.symbol, sp.closing_price
    if closing_price > max_prices[symbol]:
        max_prices[symbol] = closing_price
```

Manipuler les données

Les cours étant des tuples, ils sont triés selon leurs champs dans l'ordre :

1. en premier par sigle,
2. puis par date,
3. puis par cours.

En d'autres termes, si certains cours s'affichent tous avec le même sigle, sorted les trie par date (puis par cours, ce qui revient à ne rien faire puisque nous n'en avons qu'un par date) ; c'est ce que nous voulons.

```
# trier les cours par date
prices = {symbol: sorted(symbol_prices)
    for symbol, symbol_prices in prices.items()}
```

Manipuler les données

Nous nous en servons pour calculer une séquence de variations au jour le jour :

```
def pct_change(yesterday: StockPrice, today: StockPrice) -> float:
    return today.closing_price / yesterday.closing_price - 1

class DailyChange(NamedTuple):
    symbol: str
    date: datetime.date
    pct_change: float

def day_over_day_changes(prices: List[StockPrice]) -> List[DailyChange]:
    """considère que les cours pour chaque titre sont dans l'ordre"""
    return [DailyChange(symbol=today.symbol,
            date=today.date,
            pct_change=pct_change(yesterday, today))
            for yesterday, today in zip(prices, prices[1:])]
```

Manipuler les données

Puis, nous les collectons en totalité :

```
all_changes = [change
       for symbol_prices in prices.values()
       for change in day_over_day_changes(symbol_prices)]
```

À ce stade, il est facile de trouver le plus grand et le plus petit, mais je vous laisse le trouver ;) EXERCICE

Changer d’échelle

De nombreuses techniques sont sensibles à l’échelle utilisée pour vos données.

Par exemple, étant donnés la taille et le poids de centaines de data scientists, supposez que vous essayiez de constituer des groupes (clusters) selon leurs mensurations.

Intuitivement, nous aimerions avoir des groupes qui représentent des points proches les uns des autres,

<=> il faut mesurer d’une manière ou d’une autre la distance entre les points.

Changer d’échelle

Une approche naturelle consiste à considérer les paires (taille, poids) comme des points dans un espace à deux dimensions. Prenons les personnes répertoriées dans le tableau

Personne	Taille (pouces)	Taille (cm)	Poids (livres)
A	63	160	150
B	67	170.2	160
C	70	177.8	171

Changer d’échelle

Si nous mesurons la taille en pouces, alors B a A pour voisin le plus proche :

```
from scratch.linear_algebra import distance
a_to_b = distance([63, 150], [67, 160]) # 10.77
a_to_c = distance([63, 150], [70, 171]) # 22.14
b_to_c = distance([67, 160], [70, 171]) # 11.40
```

Cependant, si nous mesurons la taille en centimètres, alors c’est désormais C qui est le plus proche de B :

```
a_to_b = distance([160, 150], [170.2, 160]) # 14.28
a_to_c = distance([160, 150], [177.8, 171]) # 27.53
b_to_c = distance([170.2, 160], [177.8, 171]) # 13.37
```

Changer d’échelle

Évidemment, le fait qu’un simple changement d’unité change le résultat pose problème. Pour cette raison, quand les dimensions ne sont pas comparables entre elles, nous redimensionnons nos données de sorte que chaque dimension ait une espérance de 0 et un écart-type de 1.

Cela permet de se débarrasser efficacement de la nature des unités en convertissant chaque dimension en « écart-type par rapport à la moyenne ».

Changer d’échelle

Pour commencer, il faut calculer l’espérance et l’écart-type (standard_deviation) de chaque dimension

```
from typing import Tuple
from scratch.linear_algebra import vector_mean
from scratch.statistics import standard_deviation

def scale(data: List[Vector]) -> Tuple[Vector, Vector]:
    """retourne l'espérance et l'écart-type de chaque colonne"""
    dim = len(data[0])
    means = vector_mean(data)
    stdevs = [standard_deviation([vector[i] for vector in data])
    for i in range(dim)]
    return means, stdevs
    vectors = [[-3, -1, 1], [-1, 0, 1], [1, 1, 1]]

means, stdevs = scale(vectors)
assert means == [-1, 0, 1]
assert stdevs == [2, 1, 0]
```

Changer d’échelle

Ensuite, nous allons les utiliser pour créer un nouvel ensemble de données :

```
def rescale(data: List[Vector]) -> List[Vector]:
    """Change l’échelle des entrées, de sorte que chaque position ait pour
    espérance 0 et pour écart-type 1.
    Ignore les positions dont l’écart-type est de 0."""
    dim = len(data[0])
    means, stdevs = scale(data)

    # faire une copie de chaque vecteur
    rescaled = [v[:] for v in data]
    for v in rescaled:
        for i in range(dim):
            if stdevs[i] > 0:
            v[i] = (v[i] - means[i]) / stdevs[i]
    return rescaled
```

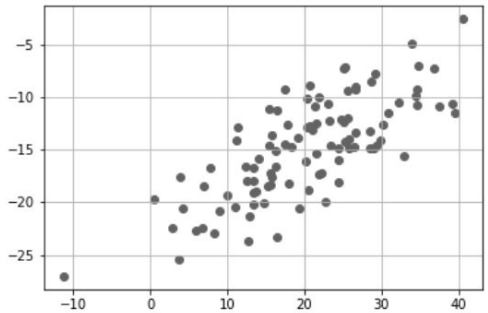
Changer d’échelle

Bien sûr, nous allons rédiger un test pour nous assurer que ce changement d’échelle produit bien les effets escomptés

```
means, stdevs = scale(rescale(vectors))
assert means == [0, 0, 1]
assert stdevs == [1, 1, 0]
```

Réduire la dimension

Parfois, les dimensions « effectives » (ou utiles) des données ne correspondent pas à celles dont nous disposons. Par exemple



L’essentiel de la variation des données semble exister pour une dimension unique qui ne correspond ni à l’axe des x, ni à celui des y

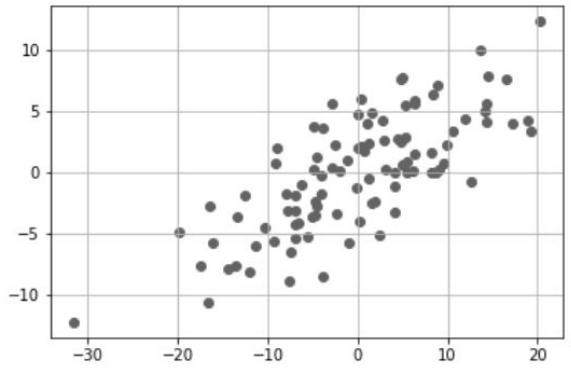
Face à un tel cas, nous pouvons utiliser la technique dite « analyse en composantesprincipales » (Principal Component Analysis – PCA) pour extraire une ou plusieurs dimensions afin de capturer la plus grande partie de la variation des données.

Changer d’échelle

La première étape consiste à transformer les données de manière à ce que chaque dimension ait pour espérance zéro

(si nous ne le faisions pas, nos méthodes risqueraient d’identifier l’espérance elle-même, plutôt que la variation de données)

```
from scratch.linear_algebra import subtract
def de_mean(data: List[Vector]) -> List[Vector]:
    """Recentre les données pour qu’elles présentent une espérance de 0
    dans chaque dimension."""
    mean = vector_mean(data)
    return [subtract(vector, mean) for vector in data]
```



Apprentissage automatique

Maschinelles Lernen

Modélisation

Qu’est-ce que l’apprentissage automatique ?

Surajustement et sous-ajustement

Exactitude

Dans mon temps libre je m’intéresse à la médecine.

J’ai développé un test non-invasif de détection des leucémies non invasif, très simple avec 98% de précision.

Comme il n’est pas brevetable je vous le partage

Si vous vous appelez Luc

Alors vous avez le cancer

Exactitude

Néanmoins, il s'agit d'un test incroyablement superficiel et donc d'un bon exemple de la raison pour laquelle nous ne spécifions généralement pas la précision comme un bon critère pour un modèle (de classification binaire)

Imaginez un modèle effectuant une classification binaire. Ce courriel est-il un spam ? Devrions-nous embaucher ce candidat ? Ce passager est-il un terroriste ?

Exactitude

À l'aide d'un ensemble de données d'entraînement contenant les réponses correctes et d'un modèle prédictif, chaque donnée est classée dans l'une des quatre catégories suivantes :

- Vrai positif : Ce message est un spam, et nous l'avons correctement identifié comme tel.

- Faux positif : Ce message n'est pas un spam, mais nous l'avons identifié comme tel.

- Faux négatif : Ce message est un spam, mais nous ne l'avons pas identifié comme tel.

- Vrai négatif : Ce message n'est pas un spam, et nous l'avions correctement prédit.

Exactitude

Ces catégories sont sont souvent représentées sous forme de fréquences dans une matrice de confusion :

	Spam	Pas Spam
Classifié spam	# de Vrais positifs	# de Faux positifs
Classifié non spam	# de Faux négatifs	# de Vrais négatifs

Exactitude

Voyons voir comment mon test de leucémie s'intègre dans ce schéma.

Aujourd'hui, environ 5 bébés sur 1000 sont prénommés Luc.

La prévalence e la leucémie touche environ 1,4%, soit 14 personne sur 1000.

Si nous supposons que ces deux facteurs sont indépendants l'un de l'autre et que nous appliquons mon test « Luke signifie leucémie » à un million de personnes, la matrice de confusion ressemblerait à ceci.

	Leucémie	Pas de l	Total
Luke	70	4930	5000
Pas Luke	13930	981070	99500
Total	14000	98600	1000000

Exactitude

Cela permet de calculer diverses statistiques sur la précision des prédictions. Par exemple, la précision (Accuracy) est définie comme la proportion de prédictions correctes.

```python
def accuracy(tp: int, fp: int, fn: int, tn: int) -> float:
    correct = tp + tn
    total = tp + fp + fn + tn
    return correct / total

assert accuracy(70, 4930, 13930, 981070) == 0.98114
```

Ce chiffre semble très impressionnant. Cependant, il ne s'agit certainement pas d'un test fiable, il ne faut donc pas se fier uniquement à sa précision.

Exactitude

En général, on considère une combinaison de précision (Precision) et de sensibilité (Recall). La précision mesure la fiabilité de nos prédictions positives.

La sensibilité mesure la proportion de cas positifs identifiés par le modèle.

```
def precision(tp: int, fp: int, fn: int, tn: int) -> float:
    return tp / (tp + fp)

assert precision(70, 4930, 13930, 981070) == 0.014




def recall(tp: int, fp: int, fn: int, tn: int) ->
float:
    return tp / (tp + fn)

assert recall(70, 4930, 13930, 981070) == 0.005
```

Exactitude

Parfois, la précision et la sensibilité sont combinées dans la mesure F1, qui est définie comme suit.

```
def f1_score(tp: int, fp: int, fn: int, tn: int) -> float:
    p = precision(tp, fp, fn, tn)
    r = recall(tp, fp, fn, tn)

    return 2 * p * r / (p + r)
```

C’est la moyenne harmonique.

Le choix d'un modèle implique généralement un compromis entre précision et sensibilité.

- Un modèle qui répond « oui » au moindre indice = sensibilité élevée mais faible précision
- un modèle qui ne répond « oui » que lorsqu'il est certain = faible sensibilité et une précision élevée.

On peut aussi y voir un compromis entre les FP et les FN :

- Dire « oui » trop souvent entraîne davantage de faux positifs
- dire « non » trop souvent entraîne de nombreux faux négatifs.

Le compromis entre le biais et la variance

Extraction et sélection des variables

Les k plus proches voisins

Classification naïve bayésienne

Régression linéaire simple

Modèle

Comme vous vous en souvenez, nous recherchions la relation entre le nombre d'amis d'un utilisateur de DataSciencester et le temps qu'il passe sur le site chaque jour. Supposons que vous soyez convaincu qu'avoir plus d'amis conduit à passer plus de temps sur le site, plutôt qu'une autre des explications dont nous avions discuté.

La responsable Engagements vous demande de construire un modèle pour décrire cette rela- tion. Comme vous avez trouvé une relation linéaire forte, le plus naturel est de construire un modèle linéaire. En particulier, vous faites l'hypothèse qu'il existe des constantes α et β telles que :

$$y_i = \beta x_i + \alpha + \varepsilon_i$$

Modèle

Supposons qu'on se fixe un α et un β donnés ; on peut alors faire des prédictions de manière simple avec la fonction suivante

```
def predict(alpha: float, beta: float, x_i: float) -> float:
    return bêta * x_i + alpha
```

Toutefois, comment choisir α et β ? Chaque choix nous permet de prédire un résultat pour chaque entrée x_i. Comme nous connaissons le résultat réel y_i, nous pouvons calculer l'erreur pour chaque paire :

```
def erreur(alpha: float, beta: float, x_i: float, y_i: float) -> float:
    """L'erreur de la prédiction beta * x_i + alpha
    quand la valeur réelle est y_i."""
    return predict(alpha, beta, x_i) - y_i
```

Modèle

Nous aimerions connaître l'erreur totale sur l'ensemble du jeu de données. Cependant, nous ne voulons pas simplement additionner les erreurs : si la prédiction pour x_1 est trop élevée et celle pour x_2 trop faible, les erreurs pourraient s'annuler.

Donc, à la place nous ajoutons les erreurs au carré :

```
from scratch.linear_algebra import Vector

def sum_of_sqerrors(alpha: float, beta: float,
                    x: Vector, y: Vector) -> float:
    return sum(error(alpha, beta, x_i, y_i) ** 2
               for x_i, y_i in zip(x, y))
```

Modèle

La solution des moindres carrés consiste à choisir α et β pour rendre sum_of_sqerrors la plus petite possible.

En faisant appel au calcul (ou à une algèbre fastidieuse), on obtient les valeurs α et β qui mini- misent l'erreur :

```python
from typing import Tuple
from scratch.linear_algebra import Vector
from scratch.statistics import correlation, standard_deviation, mean

def least_squares_fit(x: Vector, y: Vector) -> Tuple[float, float]:
    """Soit deux vecteurs x et y,
    trouver les valeurs de moindres carrés de alpha et bêta."""
    beta = correlation(x, y) * standard_deviation(y) / standard_deviation(x)
    alpha = mean(y) - beta * mean(x)
    return alpha, beta
```

Modèle

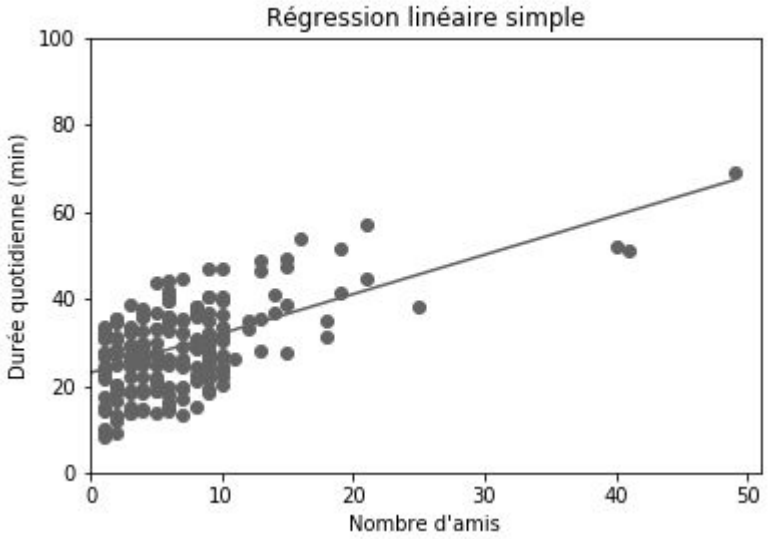
Test

```
x = [i for i in range(-100, 110, 10)]
y = [3 * i - 5 for i in x]
# doit trouver que y = 3x - 5
assert least_squares_fit(x, y) == (-5, 3)
```

On peut facilement appliquer ces principes aux données du chapitre 5 (en excluant les valeurs extrêmes)

```
from scratch.statistics import num_friends_good, daily_minutes_good
alpha, beta = least_squares_fit(num_friends_good, daily_minutes_good)
assert 22.9 < alpha < 23.0
assert 0.9 < beta < 0.905
```

Modèle



Il est évident qu'il nous faut une meilleure vérification de la bonne adéquation du modèle que celle obtenue par simple examen visuel du graphique.

Modèle

Une mesure classique est donnée par le coefficient de détermination R^2. Il mesure la fraction de la variation totale de la variable dépendante capturée par le modèle

```
from scratch.statistics import de_mean

def total_sum_of_squares(y: Vector) -> float:
    """variation totale du carré des y_i à partir de leur espérance"""
    return sum(v ** 2 for v in de_mean(y))

def r_squared(alpha: float, beta: float, x: Vector, y: Vector) -> float:
    """Fraction de la variation de y capturée par le modèle, qui est égale
    à 1 - la fraction de variation de y non capturée par le modèle."""
    return 1.0 - (sum_of_sqerrors(alpha, beta, x, y)/
total_sum_of_squares(y))
    rsq = r_squared(alpha, beta, num_friends_good, daily_minutes_good)
    assert 0.328 < rsq < 0.330
```

Utiliser la descente de gradient

Si nous posons θ = [α,β], nous pouvons résoudre notre problème par la descente de gradient :

```
# nous devrions obtenir pratiquement les mêmes résultats
alpha, beta = guess
assert 22.9 < alpha < 23.0
assert 0.9 < beta < 0.905
```

```
import random
import tqdm

from scratch.gradient_descent import gradient_step
num_epochs = 10000
random.seed(0)
# choisir une valeur aléatoire pour démarrer
guess = [random.random(), random.random()]
learning_rate = 0.00001

with tqdm.trange(nb_étapes) as t:
    for _ in t:
    alpha, beta = guess
    # dérivée partielle de perte par rapport à alpha
    grad_a = sum(2 * error(alpha, beta, x_i, y_i)
                 for x_i, y_i in zip(num_friends_good, daily_minutes_good))
    # dérivée partielle de perte par rapport à bêta
    grad_b = sum(2 * error(alpha, beta, x_i, y_i) * x_i
                 for x_i, y_i in zip(num_friends_good, daily_minutes_good))
    # calculer la perte pour coller à la description tqdm
    loss = sum_of_sqerrors(alpha, beta, num_friends_good,
daily_minutes_good)
    t.set_description(f"loss: {loss:.3f}")
    # enfin actualiser la prévision
    guess = gradient_step(guess, [grad_a, grad_b], -learning_rate)
```

Estimer le maximum de vraisemblance

Une des justifications réside dans l'estimation du ***maximum de vraisemblance***. Imaginons un échantillon de données v1, …, vn qui provient d'une distribution dépendante d'un paramètre inconnu θ :

$$p(v1, ..., vn|\theta)$$

Si nous ignorons θ, nous pouvons l'approcher et le concevoir comme la vraisemblance de θ selon l'échantillon :

$$L(\theta|v1, ..., vn)$$

Estimer le maximum de vraisemblance

Le θ le plus probable = la valeur qui maximise la fonction de vraisemblance ;

⇔ la valeur qui rend la donnée observée la plus probable.

Dans le cas d'une distribution continue, pour laquelle nous disposons d'une fonction de distribution de probabilités plutôt que d'une fonction de masse probable, nous pouvons faire la même chose.

Estimer le maximum de vraisemblance

Revenons maintenant à la régression.

Une hypothèse fréquente concernant le modèle de régression simple :

les erreurs de régression suivent une loi normale d'espérance 0 et d'écart-type (connu) σ.

Dans ce cas, la fonction de vraisemblance de l'apparition d'une paire (x_i,y_i) devient

L(\alpha,\beta|x_i,y_j,\sigma)=\frac{1}{\sigma\sqrt{2\pi}}\exp(-\frac{y_i-a-\beta x_i}{2\sigma²})

15. Régression multiple

16. Arbres de décision

17. Réseaux neuronaux

18. Apprentissage profond

Nuages de mots

nous avons calculé des nombres de mots marquant les intérêts des utilisateurs.

Une méthode pour visualiser les mots et leur nombre d’occurrences consiste à dessiner des nuages de mots dans lesquels ces derniers sont affichés dans une taille proportionnelle à leur fréquence.

Perceptron

Un neurone unique à n entrées binaires. Il calcule une somme pondérée de ses entrées et « se déclenche » si la somme vaut zéro ou davantage

```
from scratch.linear_algebra import Vector, dot

def step_function(x : float) -> float :
    return 1.0 if x >= 0 else 0.0

def perceptron_output(weights : Vector,
                      bias : float, x : Vector) -> float :
    """retourne 1 si le perceptron se déclenche, 0 sinon"""
    calculation = dot(weights, x) + bias
    return step_function(calculation)
```

Le perceptron fait simplement la distinction entre les demi-espaces séparés par l'hyperplan des points x pour lesquels `dot(weights, x) + bias == 0`

Lorsqu’on choisit correctement les pondérations,

les perceptrons savent résoudre un grand nombre de problèmes simples

Modèle

Par exemple, nous pouvons créer une *porte logique* `and` (qui renvoie 1 si les deux entrées valent 1, mais 0 si une des entrées vaut 0)

```
and_weights = [2., 2]
and_bias = -3.
assert perceptron_output(and_weights, and_bias, [1, 1]) == 1
assert perceptron_output(and_weights, and_bias, [0, 1]) == 0
assert perceptron_output(and_weights, and_bias, [1, 0]) == 0
assert perceptron_output(and_weights, and_bias, [0, 0]) == 0
```

EXERCICE : Selon un raisonnement similaire, créez une porte logique or et une porte not (à une entrée, qui convertit 0 en 1 et 1 en 0)

Malgré tous vos efforts, aucun perceptron ne saura construire une porte logique xor

il existe des problèmes qu'il est tout simplement impossible de résoudre par un perceptron unique.

EXERCICE : créé une porte logique xor qui renvoit 1 si une seule des entrées vaut 1 et 0

Réseaux neuronaux à propagation vers l'avant

On parle alors de

1. couche d'entrée (qui reçoit les entrées et les transmet vers l'avant sans les changer),
2. d'au moins une couche « cachée » (recevant les sorties de la couche précédente, effectuant des calculs et transmettant le résultat à la couche suivante)
3. et d'une couche de sortie (qui produit les sorties finales).

Comme dans le perceptron, chaque neurone (autre qu'entrée) présente **un poids** associé à chacune de ses entrées et **un biais**.
Pour faciliter la représentation, **ajouter le biais à la fin de notre vecteur de poids**
et donner à **chaque neurone un biais d'entrée toujours égal à 1.**

Réseaux neuronaux à propagation vers l'avant

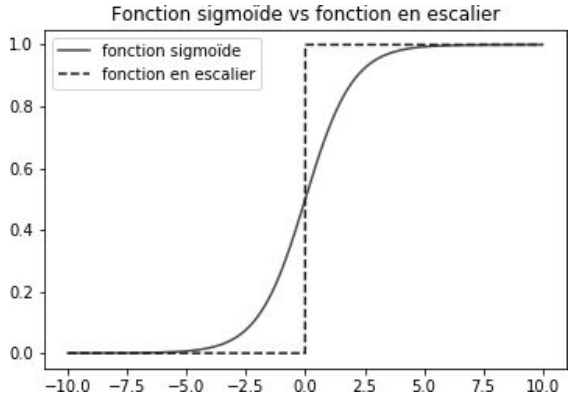
Pour chaque neurone, nous calculerons

la somme des produits de ses entrées et de ses poids

comme avec le perceptron.

Cependant, plutôt que d'appliquer step_function (fonction en escalier) à cette somme pour générer une sortie, nous en retournerons une approximation lissée. Nous utiliserons la fonction sigmoïde

```
def neuron_output(weights: Vector, inputs:
Vector) -> float:
    # le poids (weights) inclut le terme de
biais, l'entrée (inputs) inclut un 1
    return sigmoid(dot(weights, inputs))
```



Réseaux neuronaux à propagation vers l'avant

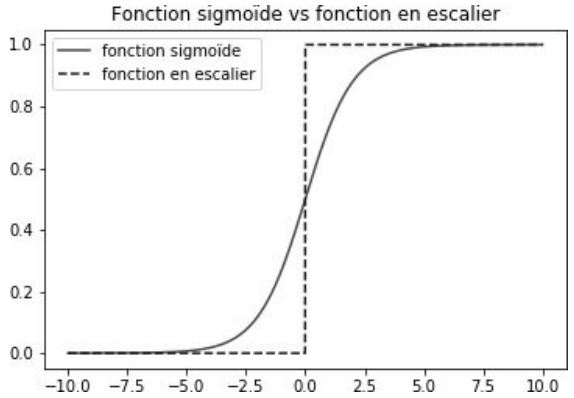
Pour chaque neurone, nous calculerons

la somme des produits de ses entrées et de ses poids

comme avec le perceptron.

Cependant, plutôt que d'appliquer step_function (fonction en escalier) à cette somme pour générer une sortie, nous en retournerons une approximation lissée. Nous utiliserons la fonction sigmoïde

```
def neuron_output(weights: Vector, inputs:
Vector) -> float:
    # le poids (weights) inclut le terme de
biais, l'entrée (inputs) inclut un 1
    return sigmoid(dot(weights, inputs))
```



Étant donné sigmo,
neurone **= vecteur de poids**
dont la longueur est supérieure de un au nombre d’entrées

Ensuite, nous représentons un réseau neuronal comme une liste de couches (autres que couche d’entrée), dont chacune est une liste des neurones qu’elle contient.

Perceptron

Donc nous représentons un réseau neuronal comme une liste (couches) de listes (neurones) de vecteurs (poids).

```
from typing import List

def feed_forward(neural_network: List[List[Vector]],
input_vector: Vector) -> List[Vector]:
    """Fournit le vecteur d'entrée via le réseau neuronal.
    Renvoie les sorties de toutes les couches (pas uniquement
    de la dernière)."""
    outputs: List[Vector] = []
    for layer in neural_network:
        input_with_bias = input_vector + [1] # ajouter une constante
        output = [neuron_output(neuron, input_with_bias) # calculer la sortie
                  for neuron in layer] # pour chaque neurone
    outputs.append(output) # ajouter les résultats
    # l'entrée de la couche suivante est la sortie de cette couche-ci
    input_vector = output
    return outputs
```

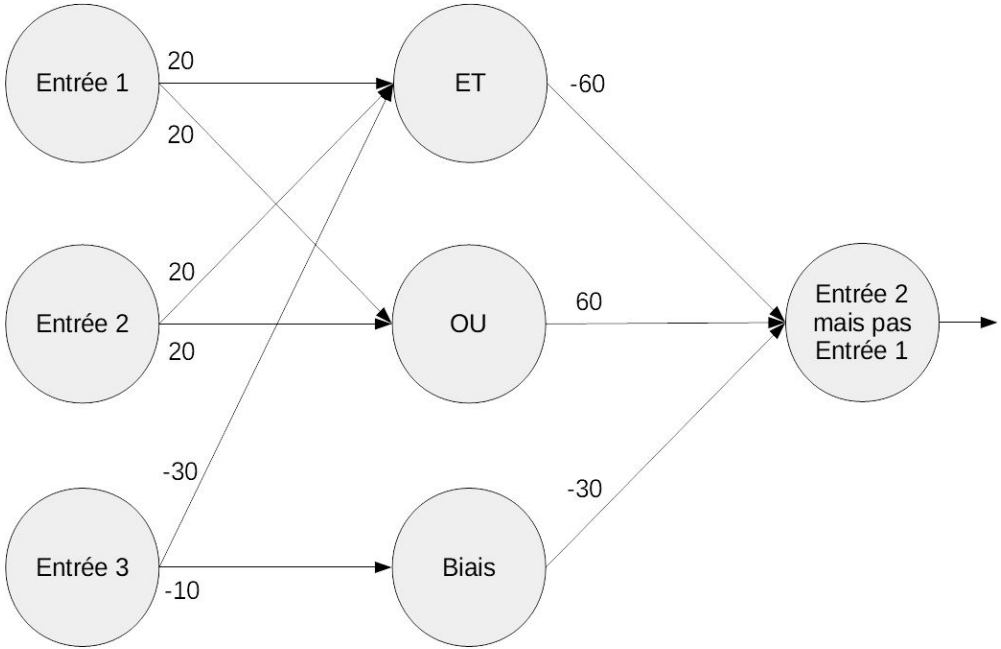
Perceptron

Il devient alors facile de construire la porte xor . Il suffit pour cela de grossir l'échelle des poids, de telle sorte que les résultats neu- ronaux (neuron_outputs) soient très proches de 0 ou de 1

```
xor_network = [# couche cachée
[[20., 20, -30], # neurone et
[20., 20, -10]], # neurone ou
# couche de sortie
[[-60., 60, -30]]] # neurone "2e entrée mais pas 1re"
# la propagation vers l'avant produit les sorties de toutes les couches
# Le [-1] obtient la sortie finale
# et le [0] la valeur sortant du vecteur résultant.
assert 0.000 < feed_forward(xor_network, [0, 0])[-1][0] < 0.001
assert 0.999 < feed_forward(xor_network, [1, 0])[-1][0] < 1.000
assert 0.999 < feed_forward(xor_network, [0, 1])[-1][0] < 1.000
assert 0.000 < feed_forward(xor_network, [1, 1])[-1][0] < 0.001
```

Perceptron

Il devient alors facile de construire la porte xor . Il suffit pour cela de grossir l'échelle des poids, de telle sorte que les résultats neuronaux (neuron_outputs) soient très proches de 0 ou de 1



Rétropropagation

Les réseaux neuronaux ne sont généralement pas créés à la main.

- nous les utilisons pour résoudre des problèmes particulièrement complexes : un problème de reconnais- sance d'image peut impliquer des centaines ou des milliers de neurones.
- nous ne pouvons le plus souvent pas « déduire » ce que devraient être les neurones.

À la place, nous utilisons (comme d'habitude) des données pour entraîner un réseau neuronal.

L'outil le plus répandu, pour cette tâche, est **l'algorithme de rétropropagation**, qui fait appel à la descente de gradient ou à une de ses variantes

Rétropropagation

Imaginons un jeu de données d'apprentissage composé de vecteurs d'entrée et des vecteurs de sortie correspondants.

Par exemple, dans notre xor_network précédent, le vecteur d'entrée `[1, 0]` correspondait à la sortie cible `[1]`. Imaginons que notre réseau comporte un ensemble de poids.

Nous allons ajuster les poids à l'aide de l'algorithme suivant :

1. **Exécuter** feed_forward sur un vecteur d'entrée pour produire les sorties de tous les neurones du réseau.
2. Nous connaissons la sortie cible, aussi nous pouvons **calculer une perte** qui correspond à la somme des erreurs au carré.
3. **Calculer le gradient** de cette perte sous la forme d'une fonction des poids du neurone de sortie.
4. « **Propager** » les gradients et erreurs vers l'arrière pour calculer les gradients par rapport aux poids des neurones cachés.
5. **Effectuer un pas** de descente de gradient.

Traitement automatique du langage naturel

Nuages de mots

Par exemple, imaginons que, pour chaque collection de mots à la mode relatifs à la data science, vous ayez deux nombres entre 0 et 100 : le premier représente

la fréquence du mot dans les offres d'emploi, le second sa fréquence dans les CV :

```
data = [ ("big data", 100, 15), ("Hadoop", 95, 25),
       ("Python", 75, 50),
       ("R", 50, 40), ("machine learning", 80, 20),
       ("statistics", 20, 60),("data science", 60, 70),
       ("analytics", 90, 3),("team player", 85, 85),
       ("dynamic", 2, 90), ("synergies", 70, 0),
       ("actionable insights", 40, 30),
       ("think out of the box", 45, 10),
       ("self-starter", 30, 50),
       ("customer focus", 65, 15),("thought leadership", 35, 35)]
```

Nuages de mots

On va répartir les mots de sorte que

- la position horizontale reflète la popularité dans les offres
- et la position verticale celle dans les CV

```
from matplotlib import pyplot as plt

def text_size(total: int) -> float:
    """égale 8 si total vaut 0, 28 si total égale 200"""
    return 8 + total / 200 * 20

for word, job_popularity, resume_popularity in data:
    plt.text(job_popularity, resume_popularity, word,
    ha='center', va='center',
    size=text_size(job_popularity + resume_popularity))
    plt.xlabel("Popularité dans les offres")
    plt.ylabel("Popularité dans les CV")
    plt.axis([0, 100, 0, 100])
    plt.xticks([])
    plt.yticks([])
    plt.show()
```

Nuages de mots

On va répartir les mots de sorte que

- la position horizontale reflète la popularité dans les offres
- et la position verticale celle dans les CV

```
from matplotlib import pyplot as plt

def text_size(total: int) -> float:
    """égale 8 si total vaut 0, 28 si total égale 200"""
    return 8 + total / 200 * 20

for word, job_popularity, resume_popularity in data:
    plt.text(job_popularity, resume_popularity, word,
    ha='center', va='center',
    size=text_size(job_popularity + resume_popularity))
    plt.xlabel("Popularité dans les offres")
    plt.ylabel("Popularité dans les CV")
    plt.axis([0, 100, 0, 100])
    plt.xticks([])
    plt.yticks([])
    plt.show()
```


dynamique
esprit d'équipe
science des données
statistique
esprit d'initiative
Python
R
meneur d'équipe'
idées constructives
Hadoop
apprentissage machine
besoins du client
big data
créativité
analyse
synergies
Popularité dans les CV
Popularité dans les offres

Modèles n-grammes

La responsable Marketing et SEO de DataSciencester **veut créer des milliers de pages web sur le thème de la data science** afin que le site bénéficie d'un meilleur classement dans les résultats de recherche sur les termes relatifs à ce sujet.

Évidemment, elle n'a aucune intention d'écrire des milliers de pages web ni de payer une armée de spécialistes de la stratégie de contenu pour le faire. À la place, elle vous demande si vous pouvez générer ce genre de pages automatiquement, à l'aide d'un programme. Pour cela, vous devrez trouver un moyen de modéliser le langage.

Une méthode consiste à utiliser un corpus de documents pour entraîner un modèle statistique du langage.

Modèles n-grammes

Tout d’abord, les apostrophes dans le texte sont représentées par le caractère Unicode u"\u2019".

Nous allons créer une fonction support pour les remplacer par des apostrophes normales :

```
def fix_unicode(text: str) -> str:
    return text.replace(u"\u2019", "'")
```

Le second problème est que, une fois que nous avons récupéré le texte de la page web, il faut le diviser en séquence de mots et de points (pour pouvoir signifier les endroits où se terminent les phrases).

Pour ce faire, nous utilisons *re.findall*

Modèles n-grammes

```
import re
from bs4 import BeautifulSoup
import requests

url = "https://www.oreilly.com/ideas/what-is-data-science"
html = requests.get(url).text
soup = BeautifulSoup(html, 'html5lib')

# trouver les div de l'article
content = soup.find("div", "main-post-radar-content")
regex = r"[\w']+|[\.]" # correspond à un mot ou un point
document = []

for paragraph in content("p"):
    words = re.findall(regex, fix_unicode(paragraph.text))
    document.extend(words)
```

Maintenant que nous avons le texte sous forme d'une séquence de mots, nous pouvons modéliser le langage de la manière suivante :

1. à partir d'un mot de départ (comme book), nous regardons tous les mots qui le suivent dans le document source.
2. Nous choisissons un de ces mots au hasard pour être le mot suivant et nous répétons ce processus jusqu'à ce que nous rencontrions un point, qui spécifie la fin de la phrase.

Nous appelons cela un **modèle bigramme**, car il est complètement déterminé par les fréquences des bigrammes (les paires de mots) dans les données d'origine.

Modèles n-grammes

Que dire du mot de départ ? Nous pouvons simplement le prendre au hasard parmi les mots qui suivent un point. Pour commencer, faisons un précalcul des transitions. N'oublions pas que *zip* s'arrête lorsque n'importe laquelle de ses entrées est terminée. Donc, *zip(document, document[1:])* nous fournit précisément les paires d'éléments consécutifs du document

```
from collections import defaultdict

transitions = defaultdict(list)

for prev, current in zip(document, document[1:]):
    transitions[prev].append(current)
```

Nous sommes prêts à générer des phrases

Modèles n-grammes

```
def generate_using_bigrams() -> str:
    current = "." # signifie que le mot suivant commence une phrase
    result = []

    while True:
        next_word_candidates = transitions[current] # bigrammes (current,_)
        current = random.choice(next_word_candidates) # en choisir un
        result.append(current) # l'ajouter à la fin des résultats
        if current == ".":
            return " ".join(result) # si "." c'est fini
```

Modèles n-grammes

Les phrases produites sont du charabia, mais c’est le genre de charabia que vous pourriez publier sur votre site pour avoir l’air d’un expert.

if you may know which are you want to data sort the data feeds web friend someone on trending topics as the data in Hadoop is the data science requires a book demonstrates why visualizations are but we do massive correlations across many commercial disk drives in Python language and creates more tractable form making connections then use and uses it to solve a data.

Si vous savez peut-être qui sont vous voulez données trier les données flux web amis quelqu’un sur les sujets tendances que les données dans Hadoop est la science des données exige un livre démontre pourquoi les visualisations sont, mais nous faisons des corrélations massives dans de nombreux lec- teurs de disques commerciaux en langage Python et crée plus traitables connexions sous forme de prise utilisent ensuite et l’utilise pour résoudre une donnée.

Modèles n-grammes

Nous pouvons rendre les phrases moins incompréhensibles en examinant les trigrammes, ou triplets de mots consécutifs. Plus généralement, vous pouvez utiliser des n-grammes constitués de n mots consécutifs, mais trois sont déjà suffisants pour nous. Les transitions vont dépendre des deux mots précédents :

```
trigram_transitions = defaultdict(list)
starts = []

for prev, current, next in zip(document, document[1:], document[2:]):
    if prev == ".": # si le mot précédent est un point
    starts.append(current) # alors ceci est un mot de départ
    trigram_transitions[(prev, current)].append(next)
```

Remarquez que nous devons à présent suivre les mots de départ séparément. Nous pouvons générer des phrases d’une manière très similaire :

Modèles n-grammes

```
def generate_using_trigrams() -> str:
    current = random.choice(starts) # choisir un mot de départ aléatoirement
    prev = "." # et le faire précéder d'un '.'
    result = [current]
    while True:
        next_word_candidates = trigram_transitions[(prev, current)]
        next_word = random.choice(next_word_candidates)
        prev, current = current, next_word
        result.append(current)
        if current == ".":
            return " ".join(result)
```

Modèles n-grammes

In hindsight MapReduce seems like an epidemic and if so does that give us new insights into how economies work That's not a question we could even have asked a few years there has been instrumented.

Avec le recul MapReduce est comme une épidémie et, si tel est le cas, cela nous donne de nouvelles perspectives sur la façon dont les économies fonctionnent. Ce n'est pas une question que nous aurions pu poser il y a quelques années qui a été instrumentée.)

Grammaires

approche différente de la modélisation de la langue = utiliser des grammaires,

autrement dit des règles pour générer des phrases acceptables.

À l'école primaire, vous avez sans doute appris les différentes composantes du discours et comment les combiner.

Par exemple, si votre instituteur était vraiment très mauvais, vous diriez qu'une phrase consiste obligatoirement en un nom suivi d'un verbe.

Grammaires

Comment générer des phrases à partir de cette grammaire ? Commençons par une liste

contenant la règle de phrase ["_S"]. Développons de manière répétitive chaque règle en la remplaçant par une autre choisie au hasard dans ses productions.

Nous nous arrêterons lorsque nous serons arrivés à une liste qui ne contient que des terminaisons.

```
['_S']
['_NP','_VP']
['_N','_VP']
['Python','_VP']
['Python','_V','_NP']
['Python','trains','_NP']
['Python','trains','_A','_NP','_P','_A','_N']
['Python','trains','logistic','_NP','_P','_A','_N']
['Python','trains','logistic','_N','_P','_A','_N']
['Python','trains','logistic','data science','_P','_A','_N']
['Python','trains','logistic','data science','about','_A', '_N']
['Python','trains','logistic','data science','about','logistic','_N']
['Python','trains','logistic','data
science','about','logistic','Python']
```

Grammaires

Comment mettre en œuvre ce mécanisme ? Pour commencer, nous allons créer une fonction support simple pour identifier les terminaisons :

```
def is_terminal(token: str) -> bool:
    return token[0] != "_"
```

Ensuite, nous devons écrire une fonction pour transformer une liste d'unités lexicales (token) en une phrase. Cherchons la première unité non terminale.

Si nous n'en trouvons aucune, c'est que nous sommes en présence d'une phrase complète : nous avons terminé.

Si nous trouvons une règle non terminale, nous choisissons au hasard une de ses productions.

Si cette production est une terminaison (c'est-à-dire un mot), nous remplaçons l'unité par ce mot.

Sinon, c'est une séquence d'unités non terminales séparées par des espaces que nous devons séparer et ensuite raccorder avec les unités en cours.

Dans tous les cas, nous répéterons le processus sur le nouvel ensemble d'unités

Grammaires

```
def expand(grammar: Grammar, tokens: List[str]) -> List[str]:
for i, token in enumerate(tokens):
    # s'il s'agit d'une unité terminale, l'ignorer
    if is_terminal(token): continue
    # Sinon, l'unité est non terminale,
    # donc nous devons choisir son remplaçant au hasard.
    replacement = random.choice(grammar[token])
    if is_terminal(replacement):
        tokens[i] = replacement
    else:
        # le remplacement peut être, par exemple, "_NP _VP"
        # nous devons donc le diviser sur les espaces
        tokens = tokens[:i] + replacement.split() + tokens[(i+1):]
        # appel récursif sur la nouvelle liste d'éléments lexicaux
        return expand(grammar, tokens)
    # nous n'avons plus que des terminaisons donc nous avons terminé
    return tokens
```

Grammaires

Nous pouvons commencer à créer des phrases :

```
def generate_sentence(grammar: Grammar) -> List[str]:
    return expand(grammar, ["_S"])
```

Ensuite, nous devons écrire une fonction pour transformer une liste d'unités lexicales (token) en une phrase. Cherchons la première unité non terminale.

Si nous n'en trouvons aucune, c'est que nous sommes en présence d'une phrase complète : nous avons terminé.

Si nous trouvons une règle non terminale, nous choisissons au hasard une de ses productions.

Si cette production est une terminaison (c'est-à-dire un mot), nous remplaçons l'unité par ce mot.

Sinon, c'est une séquence d'unités non terminales séparées par des espaces que nous devons séparer et ensuite raccorder avec les unités en cours.

Dans tous les cas, nous répéterons le processus sur le nouvel ensemble d'unités

Aparté : échantillonnage de Gibbs

Il est facile de générer des échantillons à partir de certaines distributions. On peut, par exemple, obtenir des variables aléatoires de distribution uniforme :

```
random.random()
```

Et des variables aléatoires de distribution normale :

```
inverse_normal_cdf(random.random())
```

Toutefois, certaines distributions sont plus difficiles à échantillonner. L'**échantillonnage de Gibbs** est une technique de génération d'échantillons à partir de distributions multidimensionnelles. Elle s'avère utile lorsqu'on ne connaît que quelques-unes des distributions conditionnelles.

Aparté : échantillonnage de Gibbs

Prenons l'exemple d'un lancer de deux dés. Soit x la valeur du premier et y la somme des deux. Imaginons que vous vouliez générer de nombreuses paires (x, y). Dans ce cas, il est facile de générer les échantillons directement :

```python
from typing import Tuple
import random

def roll_a_die() -> int:
    return random.choice([1, 2, 3, 4, 5, 6])

def direct_sample() -> Tuple[int, int]:
    d1 = roll_a_die()
    d2 = roll_a_die()
    return d1, d1 + d2
```

Aparté : échantillonnage de Gibbs

Imaginons maintenant que vous ne connaissiez que les distributions conditionnelles. La distribution de y sachant x est facile : si vous connaissez la valeur de x, y donnera, avec une probabilité égale, x + 1, x + 2, x + 3, x + 4, x + 5 ou x + 6 :

```
def random_y_given_x(x: int) -> int:
    """probabilité égale de donner x + 1, x + 2, ... , x + 6"""
    return x + roll_a_die()
```

Aparté : échantillonnage de Gibbs

C’est en revanche plus compliqué dans l’autre sens.

Par exemple, si vous savez que y vaut 2, alors nécessairement x vaut 1 (car la seule manière d’obtenir une somme égale à 2 avec deux dés est que chacun affiche 1).

Si vous savez que y est égal à 3, alors x a autant de chances d’être égal à 1 qu’à 2.

De même, si y est égal à 11, alors x doit être égal à 5 ou à 6

```python
def random_x_given_y(y: int) -> int:
    if y <= 7:
        # Si le total est 7 ou moins, le premier dé a la même probabilité
        # de valoir 1, 2, ..., (total - 1).
        return random.randrange(1, y)
    else:
        # Si le total vaut plus de 7, le premier dé a la même probabilité
        # de valoir (total - 6), (total - 5), ..., 6.
        return random.randrange(y - 6, 7)
```

Aparté : échantillonnage de Gibbs

Comment réaliser l'échantillonnage de Gibbs ?

1. Nous commençons avec une valeur quelconque (valide) pour *x* et *y*
2. et nous répétons en alternance le remplacement de x par une valeur aléatoire sachant y et le remplacement de y par une valeur aléatoire sachant *x*.
3. Après un certain nombre d'itérations, les valeurs résultantes de x et y représentent un échantillon de la distribution jointe non conditionnelle :

```
def gibbs_sample(num_iters: int = 100) -> Tuple[int, int]:
    x, y = 1, 2 # sans réelle importance
    for _ in range(num_iters):
        x = random_x_given_y(y)
        y = random_y_given_x(x)
    return x, y
```

Aparté : échantillonnage de Gibbs

Comment réaliser l'échantillonnage de Gibbs ?

1. Nous commençons avec une valeur quelconque (valide) pour *x* et *y*
2. et nous répétons en alternance le remplacement de x par une valeur aléatoire sachant y et le remplacement de y par une valeur aléatoire sachant *x*.
3. Après un certain nombre d'itérations, les valeurs résultantes de x et y représentent un échantillon de la distribution jointe non conditionnelle :

```
def gibbs_sample(num_iters: int = 100) -> Tuple[int, int]:
    x, y = 1, 2 # sans réelle importance
    for _ in range(num_iters):
        x = random_x_given_y(y)
        y = random_y_given_x(x)
    return x, y
```

Modélisation thématique

Quand nous avons mis en place la recommandation « des experts que vous connaissez peut-être » au chapitre 1, nous avons simplement recherché des correspondances exactes parmi les centres d'intérêt des utilisateurs.

Pour comprendre les motivations de nos utilisateurs, il serait plus subtil d'essayer d'identifier les thèmes sous-jacents.

La technique appelée **allocation de Dirichlet latente** (Latent Dirichlet Analysis – LDA) est couramment utilisée pour identifier des thèmes communs dans un ensemble de documents.

Nous l'appliquerons aux documents qui représentent les centres d'intérêt de chaque utilisateur.

Modélisation thématique

La LDA présente des similarités avec le classificateur de Bayes naïf que nous avons construit avant :

comme lui, elle suppose un modèle probabiliste pour les documents.

Avant de discuter des détails mathématiques les plus croustillants, voyons ce que suppose le modèle dans notre cas :

• il existe u**n nombre K fixe de thèmes** ;

• il existe **une variable aléatoire qui affecte à chaque thème une distribution probabiliste selon les mots**. Vous pouvez considérer cette distribution comme la probabilité de rencontrer un mot *m* pour le thème donné *k* ;

• il existe **une autre variable aléatoire qui affecte à chaque document une distribution probabiliste selon les thèmes**. Vous pouvez considérer cette distribution comme le mélange de thèmes dans un document *d* ;

• **chaque mot dans un document a été généré en choisissant d'abord un thème au hasard** (à partir de la distribution de probabilité des thèmes du document) **et en choisissant au hasard un mot** (à partir de la distribution de probabilité des mots du thème).

Modélisation thématique

On considère un ensemble de documents.
Chaque document est représenté simplement comme une **liste de mots**.

En plus de cela, on associe à **chaque mot** un **thème** (numéroté de `0` à `K-1`).
Cette information est stockée dans une structure appelée `document_topics`.

Concrètement :

- `documents[d][i]` désigne le *i^e mot* du *d^e document* ;
- `document_topics[d][i]` indique le *thème* à partir duquel ce mot a été généré.

Par exemple, le cinquième mot du quatrième document est `documents[3][4]`, et le thème associé à ce mot est `document_topics[3][4]`.

Modélisation thématique

📊 Ce que cela implique

- La **répartition des thèmes dans un document** est obtenue directement en comptant les thèmes de ses mots
  → elle est définie **explicitement**.
- La **répartition des mots dans un thème** est obtenue indirectement en observant quels mots sont associés à ce thème dans l'ensemble des documents
  → elle est définie **implicitement**.

🧩 Intuition à retenir

Un document est un **mélange de thèmes**,
et un thème est un **ensemble de mots qui apparaissent souvent ensemble**.

Modélisation thématique

Bien que ces thèmes ne soient que des numéros, nous pouvons leur donner des noms plus évocateurs en examinant les mots auxquels ils accordent le plus de poids. Il nous suffit pour cela de générer *document_topics*.

C’est là que l’échantillonnage de Gibbs entre en jeu.

1. affectons à chaque mot de chaque document un thème complètement au hasard.
2. parcourons chaque document mot par mot. À chaque mot de chaque document, nous attribuons un poids pour chaque thème. Ces poids dépendent de la distribution (actuelle) des thèmes dans le document et de la distribution (actuelle) des mots pour ce thème.
3. utilisons ensuite ces poids pour échantillonner un nouveau thème pour ce mot.

Si nous réitérons ce processus plusieurs fois, nous obtiendrons au final un échantillon représentatif combinant la distribution thème-mot et la distribution document-thème.

Modélisation thématique

Pour commencer, il nous faut une fonction pour choisir au hasard un index fondé sur un jeu de poids arbitraires :

```
def sample_from(weights: List[float]) -> int:
    """retourne i avec la probabilité weights[i] / sum(weights)"""
    total = sum(weights)
    rnd = total * random.random() # uniforme entre 0 et total
    for i, w in enumerate(weights):
        rnd -= w # retourne le plus petit i tel que
        if rnd <= 0: return i # weights[0] + ... + weights[i] >= rnd
```

Modélisation thématique

Test

```
from collections import Counter

# tirer 1 000 fois et compter
draws = Counter(sample_from([0.1, 0.1, 0.8])

for _ in range(1000))
    assert 10 < draws[0] < 190 # doit être ~10 % ; test vraiment peu fiable
    assert 10 < draws[1] < 190 # doit être ~10 % ; test vraiment peu fiable
    assert 650 < draws[2] < 950 # doit être ~80 % ; test vraiment peu fiable
    assert draws[0] + draws[1] + draws[2] == 1000
```

Modélisation thématique

Nos documents sont les centres d'intérêt de nos utilisateurs, ils ressemblent à ceci :

```
documents = [
    ["Hadoop", "Big Data", "HBase", "Java", "Spark", "Storm", "Cassandra"],
    ["NoSQL", "MongoDB", "Cassandra", "HBase", "Postgres"],
    ["Python", "scikit-learn", "scipy", "numpy", "statsmodels", "pandas"],
    ["R", "Python", "statistics", "regression", "probability"],
    ["machine learning", "regression", "decision trees", "libsvm"],
    ["Python", "R", "Java", "C++", "Haskell", "programming languages"],
    ["statistics", "probability", "mathematics", "theory"],
    ["machine learning", "scikit-learn", "Mahout", "neural networks"],
    ["neural networks", "deep learning", "Big Data", "artificial intelligence"],
    ["Hadoop", "Java", "MapReduce", "Big Data"],
    ["statistics", "R", "statsmodels"],
    ["C++", "deep learning", "artificial intelligence", "probability"],
    ["pandas", "R", "Python"],
    ["databases", "HBase", "Postgres", "MySQL", "MongoDB"],
    ["libsvm", "regression", "support vector machines"]
]
```

Modélisation thématique

Nous allons y rechercher K = 4 thèmes. Pour calculer les poids des échantillons, nous devons assurer le suivi de plusieurs compteurs. Créons tout d'abord les structures de données pour les accueillir.

• Nombre de fois que chaque thème est affecté à chaque document :

```
# liste de compteurs (Counter) ; un pour chaque document
document_topic_counts = [Counter() for _ in documents]
```

• Nombre de fois que chaque mot est affecté à chaque thème :

```
# liste de compteurs (Counter) ; un pour chaque thème
topic_word_counts = [Counter() for _ in range(K)]
```

• Nombre total des mots affectés à chaque thème :

```
# liste de nombres ; un pour chaque thème
topic_counts = [0 for _ in range(K)]
```

Modélisation thématique

- Nombre total de mots contenus dans chaque document :

```
# liste de nombres ; un pour chaque document
document_lengths = [len(document) for document in documents]
```

- Nombre de mots distincts :

```
distinct_words = set(word for document in documents
    for word in document)
    W = len(distinct_words)
```

- Nombre de documents :

```
D = len(documents)
```

Une fois que nous avons renseigné ces compteurs, nous pouvons, par exemple, chercher le nombre des mots de documents[3] qui sont associés avec le thème 1 :

```
document_topic_counts[3][1]
```

Modélisation thématique

Nous pouvons aussi chercher le nombre de fois où nlp est associé avec le thème 2 :

```
topic_word_counts[2]["nlp"]
```

Nous sommes enfin prêts à définir nos fonctions de probabilités conditionnelles. Comme pour Bayes, chacune a un terme de lissage qui garantit que chaque thème a une chance non nulle d'être choisi dans un document et que chaque mot a une chance non nulle d'être choisi

```
def p_topic_given_document(topic: int, d: int, alpha: float = 0,1) -> float:
    """Proportion des mots du document d
    qui sont affectés au thème (plus lissage)."""
    return ((document_topic_counts[d][topic] + alpha) /
            (document_lengths[d] + K * alpha))

def p_word_given_topic(word: str, topic: int, beta: float = 0,1) -> float:
    """Proportion des mots affectés à thème
    qui valent mot (plus lissage)."""
    return ((topic_word_counts[topic][word] + beta) /
           (topic_counts[topic] + W * beta))
```

Modélisation thématique

Nous utiliserons ces fonctions pour calculer les poids destinés à mettre à jour les thèmes :

```
topic_word_counts[2]["nlp"]
```

Nous sommes enfin prêts à définir nos fonctions de probabilités conditionnelles. Comme pour Bayes, chacune a un terme de lissage qui garantit que chaque thème a une chance non nulle d'être choisi dans un document et que chaque mot a une chance non nulle d'être choisi

```
def p_topic_given_document(topic: int, d: int, alpha: float = 0,1) -> float:
    """Proportion des mots du document d
    qui sont affectés au thème (plus lissage)."""
    return ((document_topic_counts[d][topic] + alpha) /
            (document_lengths[d] + K * alpha))

def p_word_given_topic(word: str, topic: int, beta: float = 0,1) -> float:
    """Proportion des mots affectés à thème
    qui valent mot (plus lissage)."""
    return ((topic_word_counts[topic][word] + beta) /
           (topic_counts[topic] + W * beta))
```

Modélisation thématique

Nous utiliserons ces fonctions pour calculer les poids destinés à mettre à jour les thèmes :

```
def topic_weight(d: int, word: str, k: int) -> float:
    """Soit un document et un mot de ce document,
    retourner le poids pour le thème de rang k."""
    return p_word_given_topic(word, k) * p_topic_given_document(k, d)

def choose_new_topic(d: int, word: str) -> int:
    return sample_from([topic_weight(d, word, k)
    for k in range(K)])
```

il est assez intuitif de penser que, pour un mot et son document, la probabilité de choix d'un thème quelconque dépend à la fois de la probabilité de ce thème dans le document et de celle du mot par rapport au thème.

Modélisation thématique

C’est toute la mécanique dont nous avons besoin. Commençons par affecter chaque mot à un thème au hasard en mettant à jour nos différents compteurs :

```
random.seed(0)
document_topics = [[random.randrange(K) for word in document]
for document in documents]

for d in range(D):
    for word, topic in zip(documents[d], document_topics[d]):
        document_topic_counts[d][topic] += 1
        topic_word_counts[topic][word] += 1
        topic_counts[topic] += 1
```

il est assez intuitif de penser que, pour un mot et son document, la probabilité de choix d’un thème quelconque dépend à la fois de la probabilité de ce thème dans le document et de celle du mot par rapport au thème.

Modélisation thématique

Notre but est de trouver un échantillon en jointure de la distribution thèmes-mots et de la distribution thèmes-documents.

Pour cela, nous utiliserons une forme d'échantillonnage de Gibbs qui fait appel aux probabilités conditionnelles définies précédemment :

```
import tqdm
for iter in tqdm.trange(1000):
    for d in range(D):
        for i, (word, topic) in enumerate(zip(documents[d],
        document_topics[d])):
            # Enlever ce mot-thème des compteurs
            # pour ne pas influencer les poids.
            document_topic_counts[d][topic] -= 1
            topic_word_counts[topic][word] -= 1
            topic_counts[topic] -= 1
            document_lengths[d] -= 1
            # choisir un nouveau thème en s'appuyant sur les poids
            new_topic = choose_new_topic(d, word)
            document_topics[d][i] = new_topic
            # et maintenant l'ajouter de nouveau aux compteurs
            document_topic_counts[d][new_topic] += 1
            topic_word_counts[new_topic][word] += 1
            topic_counts[new_topic] += 1
            document_lengths[d] += 1
```

Modélisation thématique

Que sont les thèmes ? Ce sont seulement des numéros 0, 1, 2 et 3. Si nous voulons leur donner des noms, il faudra le faire nous-mêmes. Voyons les cinq mots qui ont le poids le plus élevé pour chaque thème

```
for k, word_counts in enumerate(topic_word_counts):
    for word, count in word_counts.most_common():
        if count > 0:
            print(k, word, count)
```

Ca devrait sortir :

Thème 0	Thème 1	Thème 2	Thème 3
Java	R	HBase	regression
Big Data	statistics	Postgres	libsvm
Hadoop	Python	MongoDB	scikit-learn
deep learning	probability	Cassandra	apprentissage automatique
artificial intelligence	pandas	NoSQL	neural networks

Modélisation thématique

Soit :

```
topic_names = ["Big Data and programming languages",
    "Python and statistics",
    "databases",
    "machine learning"]
```

À ce stade, nous pouvons voir comment le modèle affecte des thèmes à chaque centre d'intérêt des utilisateurs :

```
for document, topic_counts in zip(documents, document_topic_counts):
    print()
    print(document)
    for topic, count in topic_counts.most_common():
        if count > 0:
            print(topic_names[topic], count)
```

Modélisation thématique

Cela donne :

```
['Hadoop', 'Big Data', 'HBase', 'Java', 'Spark', 'Storm', 'Cassandra']
Big Data and programming languages 4
databases 3

['NoSQL', 'MongoDB', 'Cassandra', 'HBase', 'Postgres']
databases 5

['Python', 'scikit-learn', 'scipy', 'numpy', 'statsmodels', 'pandas']
Python and statistics 5
machine learning 1
```

Vecteurs de mots

De très nombreuses avancées en TLN impliquent l'apprentissage profond. Dans le reste de ce chapitre, nous allons examiner deux cas faisant appel à la mécanique que nous avons développée au chapitre précédent

Une innovation importante implique une représentation des mots en tant que vecteurs de mots faiblement dimensionnés.

Ces vecteurs peuvent être comparés, ajoutés les uns aux autres, fournis à des modèles d'apprentissage automatique ou manipulés de toute autre manière qui vous intéresse. Ils affichent généralement des propriétés intéressantes.

Par exemple, des mots similaires ont tendance à présenter des vecteurs similaires.

Typiquement, le vecteur de mot pour « grand » (big) est généralement assez proche de celui pour « gros » (large).

Ainsi, un modèle fonctionnant avec des vecteurs de mots peut (dans une certaine mesure) gérer des propriétés comme la synonymie

Vecteurs de mots

Les vecteurs font souvent aussi montre de propriétés arithmétiques plaisantes.

Par exemple, dans certains modèles de ce type, si vous prenez le vecteur du mot « roi » (king),

Roi - Homme + Femme = ???

Il peut s’avérer intéressant de réfléchir à la signification de cet « apprentissage » des vecteurs de mots, même si nous n’y consacrerons pas notre temps ici.

Obtenir de tels vecteurs pour un lexique important est une entreprise difficile. Aussi, nous les apprendrons à partir d’un corpus de textes.

Vecteurs de mots

Il existe deux schémas différents. Toutefois, à un degré d'abstraction supérieur, la tâche ressemble à ce qui suit.

1. Obtenir un ensemble de textes.
2. Créer un jeu de données où l'objectif consiste à prédire un mot selon des mots proches (ou, inversement, à prédire des mots proches selon un mot donné).
3. Entraîner un réseau neuronal à être efficace dans cette tâche.
4. Récupérer les états internes du réseau neuronal entraînés en tant que vecteurs de mots.

Vecteurs de mots

Plus particulièrement – sachant que la tâche consiste à prédire un mot selon des mots proches – des mots qui interviennent dans des contextes similaires devraient présenter des états internes similaires et donc des vecteurs similaires.

Ici, nous calculerons la « similarité cosinus », qui consiste en un nombre compris entre –1 et 1 mesurant dans quelle mesure deux vecteurs pointent dans la même direction :

```
from scratch.linear_algebra import dot, Vector
import math

def cosine_similarity(v1: Vector, v2: Vector) -> float:
    return dot(v1, v2) / math.sqrt(
                         dot(v1, v1) * dot(v2, v2))

assert cosine_similarity([1., 1, 1], [2., 2, 2]) == 1, "même direction"
assert cosine_similarity([-1., -1], [2., 2]) == -1, "directions opposées"
assert cosine_similarity([1., 0], [0., 1]) == 0, "directions orthogonales"
```

Procédons à l’apprentissage de quelques vecteurs de mots pour comprendre ce fonctionnement.

Vecteurs de mots

```
colors = ["red", "green", "blue", "yellow", "black", ""]
nouns = ["bed", "car", "boat", "cat"]
verbs = ["is", "was", "seems"]
adverbs = ["very", "quite", "extremely", ""]
adjectives = ["slow", "fast", "soft", "hard"]

def make_sentence() -> str:
    return " ".join([
        "The",
        random.choice(colors),
        random.choice(nouns),
        random.choice(verbs),
        random.choice(adverbs),
        random.choice(adjectives),
        "."])

NUM_SENTENCES = 50
random.seed(0)
sentences = [make_sentence() for _ in range(NUM_SENTENCES)]
```

Vecteurs de mots

Ce code va générer de nombreuses phrases de structure similaire, mais avec des mots différents ; par exemple, The green boat seems quite slow (« le bateau vert semble assez lent »).

Sachant cette configuration, les couleurs figureront essentiellement dans des contextes « similaires », tout comme les noms et ainsi de suite. Donc, si nous affectons correctement les vecteurs de mots, les couleurs devraient bénéficier de vecteurs similaires et ainsi de suite.

Vecteurs de mots

Comme nous l'avons vu auparavant (chapitre 19), nous voulons appliquer un encodage one-hot (« un parmi n ») à nos mots, ce qui signifie que nous avons besoin de les convertir en ID.

Nous définirons donc une classe Vocabulary pour suivre cette mise en correspondance (ce mappage) mot-ID

```python
from scratch.deep_learning import Tensor

class Vocabulary:
def __init__(self, words: List[str] = None) -> None:
    self.w2i: Dict[str, int] = {} # mappage word -> word_id
    self.i2w: Dict[int, str] = {} # mappage word_id -> word
    for word in (words or []): # Si des mots sont fournis,
    self.add(word) # les ajouter.

    @property
    def size(self) -> int:
        """nombre de mots du vocabulaire"""
        return len(self.w2i)

    def add(self, word: str) -> None:
        if word not in self.w2i: # si les mots sont nouveaux
        word_id = len(self.w2i) # rechercher l'ID suivant
        self.w2i[word] = word_id # ajouter à la correspondance word -> word_id map
        self.i2w[word_id] = word # ajouter au word_id -> word map.
    def get_id(self, word: str) -> int:
        """retourner l'ID du mot (ou None)"""
        return self.w2i.get(word)

    def get_word(self, word_id: int) -> str:
        """retourner le mot ayant l'ID donné (ou None)"""
        return self.i2w.get(word_id)

    def one_hot_encode(self, word: str) -> Tensor:
        word_id = self.get_id(word)
        assert word_id is not None, f"unknown word {word}"
        return [1.0 if i == word_id else 0.0 for i in range(self.size)]
```

Vecteurs de mots

Nous devrions également élaborer des fonctions d'aide simples pour enregistrer et charger un

vocabulaire, tout comme nous l'avons fait pour nos modèles d'apprentissage profond :

```python
import json

def save_vocab(vocab: Vocabulary, filename: str) -> None:
    with open(filename, 'w') as f:
    json.dump(vocab.w2i, f) # uniquement besoin d'enregistrer w2i

def load_vocab(filename: str) -> Vocabulary:
    vocab = Vocabulary()
    with open(filename) as f:
        # charger w2i et générer i2w à partir de celui-ci
        vocab.w2i = json.load(f)
        vocab.i2w = {id: word for word, id in vocab.w2i.items()}
    return vocab
```

Vecteurs de mots

Nous allons utiliser un modèle de vecteur de mot appelé skip-gram.

Il accepte un mot en entrée & génère des proba quant à l'éventualité que d'autres mots soient proches de lui.

Nous alimenterons notre modèle au moyen de paires d'entraînement (word « mot », nearby_word « mot_proche ») et nous nous efforcerons de minimiser la perte *SoftmaxCrossEntropy*.

Concevons notre réseau neuronal.

En son cœur, nous placerons une couche d'incorporation (embedding layer), qui accepte un ID de mot en entrée et retourne un vecteur de mot.

Sous le capot, il nous suffira d'utiliser une table de consultation.

Vecteurs de mots

Nous transmettrons alors le vecteur de mot à une couche *Linear* présentant le même nombre de sorties qu'il y a de mots dans notre vocabulaire.

Comme auparavant, nous utiliserons *softmax* pour convertir ces sorties en probabilités sur nos mots proches.

À mesure que nous utiliserons la descente de gradient pour entraîner le modèle, nous actualiserons les vecteurs dans la table de consultation. Une fois l'entraînement terminé, la table de consultation nous fournira nos vecteurs de mots.

Vecteurs de mots

Créons à présent la couche d'incorporation (*embedding*). En pratique, mieux vaut incorporer des éléments autres que des mots.

Nous construirons donc une couche Embedding plus géné- rale. Nous élaborerons ultérieurement une sous-classe *TextEmbedding* spécifiquement dédiée à des vecteurs de mots.

Dans son constructeur, nous allons fournir le nombre et la dimension de nos vecteurs, ce qui permettra à la couche de créer les incorporations (qui constitueront initialement des normales aléatoires standards) :

```python
from typing import Iterable
from scratch.deep_learning import Layer, Tensor, random_tensor, zeros_like

class Embedding(Layer):
    def __init__(self, num_embeddings: int, embedding_dim: int) -> None:
        self.num_embeddings = num_embeddings
        self.embedding_dim = embedding_dim
        # Un vecteur de taille embedding_dim
        # pour chaque incorporation souhaitée.
        self.embeddings = random_tensor(num_embeddings, embedding_dim)
        self.grad = zeros_like(self.embeddings)
        # enregistrer le dernier ID entré
        self.last_input_id = None
```

Vecteurs de mots

Dans notre cas, nous n'incorporerons qu'un mot à la fois.

Toutefois, dans d'autres modèles, mieux vaudra incorporer une séquence de mots et obtenir en retour une séquence de vecteurs

(par exemple, si nous voulions entraîner le modèle CBOW).

Aussi, un substitut conceptuel consisterait à accepter des séquences d'ID de mots. Nous nous en tiendrons au « un à la fois » par souci de simplicité.

```python
from typing import Iterable
from scratch.deep_learning import Layer, Tensor, random_tensor, zeros_like

class Embedding(Layer):
    def __init__(self, num_embeddings: int, embedding_dim: int) -> None:
        self.num_embeddings = num_embeddings
        self.embedding_dim = embedding_dim
        # Un vecteur de taille embedding_dim
        # pour chaque incorporation souhaitée.
        self.embeddings = random_tensor(num_embeddings, embedding_dim)
        self.grad = zeros_like(self.embeddings)
        # enregistrer le dernier ID entré
        self.last_input_id = None

    def forward(self, input_id: int) -> Tensor:
        """Sélectionner le vecteur d'incorporation correspondant
        à l'ID d'entrée."""
        self.input_id = input_id # mémoriser pour la rétropropagation
        return self.embeddings[input_id]
```

Vecteurs de mots

Les autres méthodes intégrées fonctionneront toutes telles quelles, mais nous ajouterons deux méthodes spécifiques pour fonctionner avec du texte.

Par exemple, nous aimerions pouvoir récupérer le vecteur d'un mot donné.

Ceci ne fait pas partie de l'interface *Layer*, mais nous restons toujours libres d'ajouter quelques méthodes en plus pour des couches spécifiques selon les besoins.

```python
class TextEmbedding(Embedding):
    def __init__(self, vocab: Vocabulary, embedding_dim: int) -> None:
    # appeler le constructeur de la superclasse
    super().__init__(vocab.size, embedding_dim)
    # et y rattacher le vocabulaire
    self.vocab = vocab

    def __getitem__(self, word: str) -> Tensor:
        word_id = self.vocab.get_id(word)
        if word_id is not None:
            return self.embeddings[word_id]
        else:
            return None
```

Vecteurs de mots

Cette méthode dunder (double-underscore) nous permet de récupérer des vecteurs de mots en utilisant une indexation :

```
word_vector = embedding["black"]
```

Vecteurs de mots

Nous aimerions également que la couche d'incorporation nous révèle les mots les plus proches d'un mot donné :

Notre couche d'incorporation se contente de générer en sortie des vecteurs avec lesquels nous alimentons une couche Linear.

```python
class TextEmbedding(Embedding):
    def __init__(self, vocab: Vocabulary, embedding_dim: int) -> None:
    # appeler le constructeur de la superclasse
    super().__init__(vocab.size, embedding_dim)
    # et y rattacher le vocabulaire
    self.vocab = vocab

    def __getitem__(self, word: str) -> Tensor:
        word_id = self.vocab.get_id(word)
        if word_id is not None:
            return self.embeddings[word_id]
        else:
            return None

    def closest(self, word: str, n: int = 5) -> List[Tuple[float, str]]:
        """Retourne les n mots les plus proches
        selon une similarité cosinus."""
        vector = self[word]
        # Calculer des paires (similarity, other_word)
        # puis trier les plus similaires en premier.
        scores = [(cosine_similarity(vector, self.embeddings[i]), other_word)
        for other_word, i in self.vocab.w2i.items()]
        scores.sort(reverse=True)
        return scores[:n]
```

Vecteurs de mots

À présent, nous sommes prêts à assembler nos données d'entraînement. Pour chaque mot entré, nous choisirons comme cibles les deux mots placés à sa gauche et les deux placés à sa droite.

```
import re

# cette regex n'est pas fantastique, mais fonctionne sur nos données
tokenized_sentences = [re.findall("[a-z]+|[.]", sentence.lower())
for sentence in sentences]
```

Construisons un vocabulaire :

```
# Créer un vocabulaire (à savoir, une correspondance word -> word_id)
# à partir de notre texte.
vocab = Vocabulary(word
                   for sentence_words in tokenized_sentences
                   for word in sentence_words)
```

Vecteurs de mots

Nous pouvons à présent créer des données d'entraînement :

```
from scratch.deep_learning import Tensor, one_hot_encode
inputs: List[int] = []
targets: List[Tensor] = []
for sentence in tokenized_sentences:
    for i, word in enumerate(sentence): # Pour chaque mot
        for j in [i - 2, i - 1, i + 1, i + 2]: # accepter ls emplacements proches
            if 0 <= j < len(sentence): # qui ne sont pas hors limites
            nearby_word = sentence[j] # et les récupérer.
            # ajouter une entrée qui constitue l'ID_mot d'origine
            inputs.append(vocab.get_id(word))
            # ajouter une cible qui est le mot proche encodé un_parmi-n
            targets.append(vocab.one_hot_encode(nearby_word))
```

Vecteurs de mots

Avec la mécanique que nous avons construite, il est à présent facile de créer notre modèle :

```
from scratch.deep_learning import Sequential, Linear
random.seed(0)
EMBEDDING_DIM = 5 # semble une bonne taille
# définir la couche d'incorporation séparément, afin de la référencer
embedding = TextEmbedding(vocab=vocab, embedding_dim=EMBEDDING_DIM)
model = Sequential([
                # Soit un mot (en tant que vecteur de word_ids),
                # examiner son incorporation.
                embedding,
                # Et utiliser une couche linéaire pour calculer
                # des scores pour les "mots proches".
                Linear(input_dim=EMBEDDING_DIM,
               output_dim=vocab.size)])
```

Vecteurs de mots

En recourant à la mécanique du chapitre précédent, il est facile d'entraîner notre modèle :

```
from scratch.deep_learning import SoftmaxCrossEntropy, Momentum,\
GradientDescent
loss = SoftmaxCrossEntropy()
optimizer = GradientDescent(learning_rate=0.01)
for epoch in range(100):
    epoch_loss = 0.0
    for input, target in zip(inputs, targets):
        predicted = model.forward(input)
        epoch_loss += loss.loss(predicted, target)
        gradient = loss.gradient(predicted, target)
        model.backward(gradient)
        optimizer.step(model)
    print(epoch, epoch_loss) # Imprimer la perte
    print(embedding.closest("black")) # et aussi quelques plus proches
    print(embedding.closest("slow")) # afin de voir ce qui a été
    print(embedding.closest("car")) # appris.
```

Vecteurs de mots

Une fois le modèle entraîné, il est amusant d'explorer les mots les plus similaires :

```
pairs = [(cosine_similarity(embedding[w1], embedding[w2]), w1, w2)
            for w1 in vocab.w2i
            for w2 in vocab.w2i
            if w1 < w2]
pairs.sort(reverse=True)
print(pairs[:5])
```

Dans mon cas, cela débouche sur ce qui suit :

```
[(0.9980283554864815, 'boat', 'car'),
(0.9975147744587706, 'bed', 'cat'),
(0.9953153441218054, 'seems', 'was'),
(0.9927107440377975, 'extremely', 'quite'),
(0.9836183658415987, 'bed', 'car')]
```